Fundraising Platform — Complete .NET / ASP.NET Core Implementation Blueprint

Complete .NET 10 / ASP.NET Core 10 application blueprint for the Fundraising Platform

Complete .NET Study Map

- Solution & Clean Architecture
- C# OOP & SOLID
- Generics, Collections & LINQ
- Async / Await & Parallelism
- Dependency Injection & Lifetimes
- Program.cs, Configuration & Options
- Controllers, DTOs & Validation
- Versioning, OpenAPI & ProblemDetails
- EF Core, Dapper & Stored Procedures
- Transactions & Concurrency
- Patterns, Outbox & Idempotency
- AuthN / AuthZ, OAuth, OIDC & JWT
- HttpClient, Resilience & Caching
- Performance & Observability
- xUnit & Integration Testing
- Build Order & Final Architecture

Purpose of This Document

This document takes the three databases you designed in SQL Server:

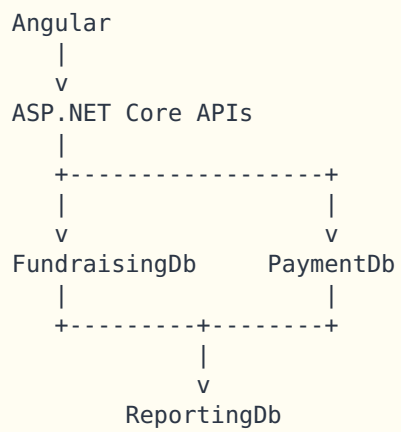
EXAMPLE / FLOW

```
FundraisingDb
PaymentDb
ReportingDb
```

and builds the .NET application layer above them.

The .NET side will eventually provide:

EXAMPLE / FLOW



This document covers:

EXAMPLE / FLOW C# fundamentals used in the project

```
OOP
interfaces
abstraction
encapsulation
inheritance
polymorphism
composition

generics
collections
LINQ

async / await
Task
Task.WhenAll
parallel programming
Parallel.ForEachAsync
thread safety

static
singleton
scoped
transient
dependency injection

configuration
Program.cs
startup concepts
Options pattern

Clean Architecture
Domain
Application
Infrastructure
API

controllers
routes
HTTP methods
DTOs
validation

API versioning
OpenAPI / Swagger

EF Core
Dapper
stored procedures
transactions
SQL Server integration

custom exceptions
global exception handling
ProblemDetails

authentication
authorization
```

```
OAuth 2.0
OpenID Connect
JWT
Microsoft Entra ID
SAML concepts
roles
claims
policies

design patterns
CQRS
Repository
Unit of Work
Factory
Strategy
Adapter
Decorator
Specification
Result
Outbox
Idempotency

background workers
HttpClient
resilience

caching
rate limiting
pagination

logging
OpenTelemetry

xUnit
unit testing
integration testing
API testing

performance for 1,000+ users
```

The most important rule is:

> **We will use a .NET feature where it solves an actual project problem. We will not force every C# feature or design pattern into every class just because it exists.**

.NET Technology Baseline

Use:

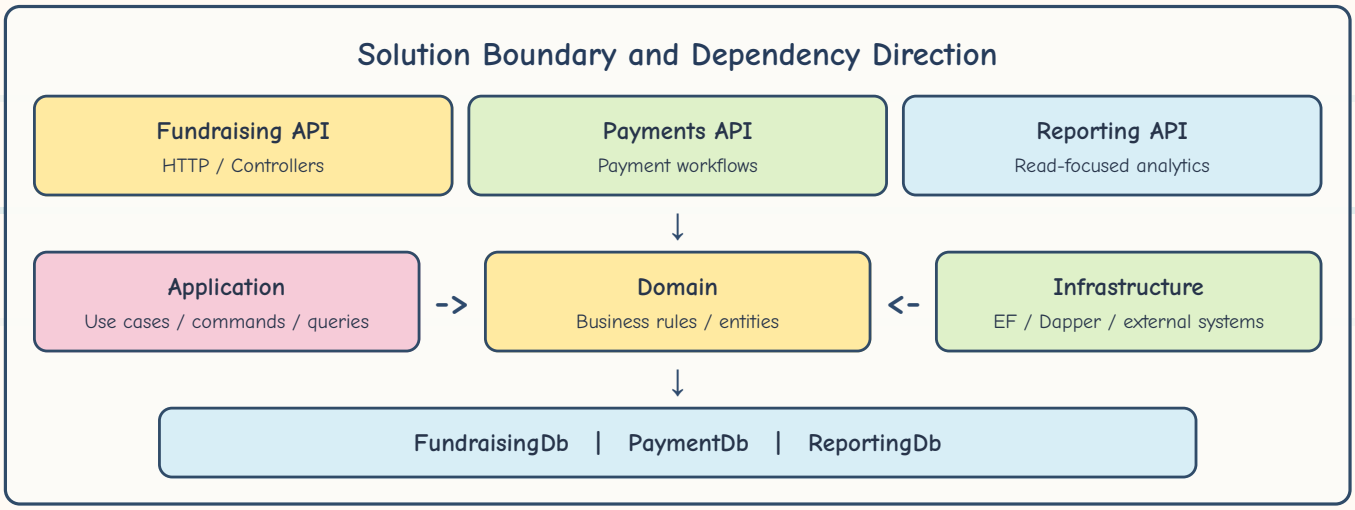
EXAMPLE / FLOW

```
.NET 10 LTS
C# 14
ASP.NET Core 10
EF Core 10
Microsoft.Data.SqlClient
xUnit
Microsoft Entra ID
```

for the learning project.

ASP.NET Core supports both controller-based APIs and Minimal APIs. For this project I recommend **controllers for the main public business APIs** because you are learning a large enterprise-style application with authentication, policies, DTO validation, API versioning, filters/conventions, and many endpoint groups. Minimal APIs remain useful for very small/internal endpoints such as health checks. ASP.NET Core's current controller model uses `ControllerBase`, `[ApiController]`, attribute routing, automatic validation behavior, and Problem Details support.

Overall .NET Solution



Create one solution:

EXAMPLE / FLOW

```
FundraisingPlatform.sln
```

Inside it:

EXAMPLE / FLOW

```
src/
│
├── BuildingBlocks/
│
├── Fundraising.Domain/
├── Fundraising.Application/
├── Fundraising.Infrastructure/
├── Fundraising.Api/
│
├── Payments.Domain/
├── Payments.Application/
├── Payments.Infrastructure/
├── Payments.Api/
│
├── Reporting.Application/
├── Reporting.Infrastructure/
├── Reporting.Api/
│
└── Gateway/                      later
```

Tests:

EXAMPLE / FLOW

```
tests/
│
├── Fundraising.Domain.Tests/
├── Fundraising.Application.Tests/
├── Fundraising.IntegrationTests/
│
├── Payments.Domain.Tests/
├── Payments.Application.Tests/
├── Payments.IntegrationTests/
│
├── Reporting.Tests/
│
└── Architecture.Tests/
```

You are keeping all of these in one solution/repository while learning.

Later, in the microservices phase:

EXAMPLE / FLOW

```
Fundraising
Payments
Reporting
```

can become independently deployed services.

• Why Separate Projects?

Because every layer has a different responsibility.

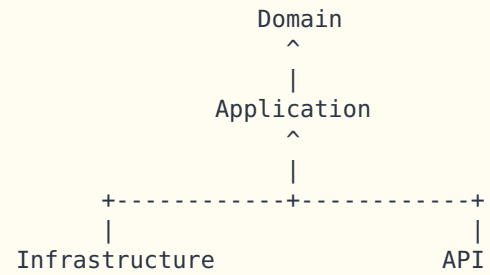
The flow should be:

EXAMPLE / FLOW

```
HTTP request
     |
     v
API
     |
     v
Application
     |
     v
Domain
     |
     v
Infrastructure
     |
     v
SQL Server / External Systems
```

More accurately, dependencies point inward:

EXAMPLE / FLOW



The Domain does not know:

EXAMPLE / FLOW

```
ASP.NET Core
EF Core
SQL Server
Azure
Angular
Service Bus
```

That separation is one of the main ideas of Clean Architecture.

• Domain Layer

The Domain contains the core business concepts.

For Fundraising:

EXAMPLE / FLOW

```
Tenant
AppUser
Constituent
Address
Campaign
Fund
Gift
GiftAllocation
Interaction
Task
```

For Payments:

EXAMPLE / FLOW

```
MerchantAccount
Payment
PaymentMethod
PaymentAttempt
FraudAssessment
Refund
Settlement
```

The domain contains:

EXAMPLE / FLOW

```
entities
value objects
domain rules
domain exceptions
domain events when useful
interfaces only when domain behavior needs them
```

It should not contain:

EXAMPLE / FLOW

```
Controllers
DbContext
SQL strings
HttpClient
Azure Service Bus
appsettings.json
```

• Application Layer

The Application layer implements **use cases**.

Examples:

EXAMPLE / FLOW

```
Create Constituent

Update Constituent

Search Constituents

Create Gift

Complete Task

Create Payment

Request Refund

Get Campaign Performance
```

It coordinates:

EXAMPLE / FLOW

```
domain objects

database abstractions

authorization context

transactions

external service abstractions
```

but does not know infrastructure implementation details.

Infrastructure Layer

Infrastructure contains technical implementations.

Examples:

EXAMPLE / FLOW

```
EF Core DbContext

SQL repositories

Dapper queries

Stored procedure calls

Payment provider adapter

Service Bus publisher

Azure Key Vault access

Outbox persistence

HttpClient implementations
```

API Layer

The API project handles HTTP.

Responsibilities:

EXAMPLE / FLOW

```
routing

controllers

authentication

authorization

request DTOs

HTTP status codes

ProblemDetails

API versioning

OpenAPI

rate limiting

CORS

middleware
```

A controller should not contain your complete Gift transaction logic.

Bad:

EXAMPLE / FLOW

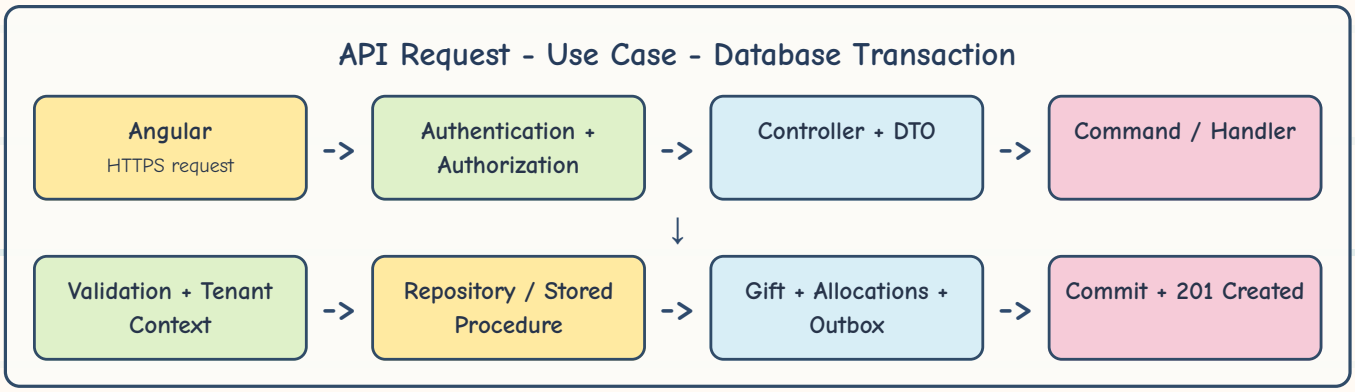
```
GiftController
    |
    + 150 lines of validation
    + SQL queries
    + transaction handling
    + JSON serialization
    + event publishing
```

Better:

EXAMPLE / FLOW

```
GiftController
       |
       v
CreateGiftCommand
       |
       v
CreateGiftHandler
       |
       v
Domain + Infrastructure
```

Complete Request Flow



Suppose Angular calls:

EXAMPLE / FLOW

```
POST /api/v1/gifts
```

Flow:

EXAMPLE / FLOW

```
Angular
   |
   | HTTPS
   v
ASP.NET Core
   |
   v
Authentication
   |
   v
Authorization
   |
   v
GiftController
   |
   v
CreateGiftCommand
   |
   v
Validation
   |
   v
CreateGiftHandler
   |
   +---- TenantContext
   |
   +---- Gift repository / stored procedure
   |
   +---- SQL transaction
   |
   +---- Gift
   |
   +---- GiftAllocation
   |
   +---- OutboxMessage
```

```
   |
   v
Commit
   |
   v
CreateGiftResult
   |
   v
HTTP 201 Created
```

That one endpoint teaches a huge portion of .NET.

C# OOP in This Project

OOP means organizing software around objects that combine:

EXAMPLE / FLOW

```
state
+
behavior
```

The important concepts are:

EXAMPLE / FLOW

```
Class
Object
Encapsulation
Abstraction
Inheritance
Polymorphism
Composition
Interface
```

Do not memorize only definitions.

Use each where appropriate.

Class and Object

A class defines a type.

Example:

EXAMPLE / FLOW

```
public sealed class Gift
{
    public long Id { get; private set; }

    public long TenantId { get; private set; }

    public long ConstituentId { get; private set; }
```

```
    public decimal TotalAmount { get; private set; }
}
```

`Gift` is the class.

An actual:

EXAMPLE / FLOW

```
GiftId = 10000
Amount = $1,000
```

is an object.

• Encapsulation

Encapsulation means an object protects its internal state and exposes controlled operations.

Bad:

EXAMPLE / FLOW

```
public class Gift
{
    public decimal TotalAmount { get; set; }

    public List<GiftAllocation> Allocations { get; set; }
}
```

Any caller can do:

EXAMPLE / FLOW

```
gift.TotalAmount = -500;
gift.Allocations.Clear();
```

without business validation.

Better:

EXAMPLE / FLOW

```
public sealed class Gift
{
    private readonly List<GiftAllocation> _allocations = [];

    public decimal TotalAmount { get; private set; }

    public IReadOnlyCollection<GiftAllocation> Allocations =>
        _allocations.AsReadOnly();

    public void AddAllocation(long fundId, decimal amount)
    {
        if (amount <= 0)
        {
            throw new InvalidAllocationException(
                "Allocation amount must be positive.");
        }
```

```
        _allocations.Add(
            new GiftAllocation(fundId, amount));
    }
}
```

Now the entity controls its own valid state.

That is encapsulation.

• Abstraction

Abstraction means exposing what callers need without exposing implementation details.

Example:

EXAMPLE / FLOW

```
public interface IPaymentGateway
{
    Task<PaymentGatewayResult> ChargeAsync(
        PaymentRequest request,
        CancellationToken cancellationToken);
}
```

Application code knows:

EXAMPLE / FLOW

```
charge payment
```

but does not need to know:

EXAMPLE / FLOW

```
Stripe HTTP URL
Adyen request JSON
certificate details
retry handler
```

That implementation belongs in Infrastructure.

• Interface

An interface defines a contract.

For example:

EXAMPLE / FLOW

```
public interface IPaymentGateway
{
    Task<PaymentGatewayResult> ChargeAsync(
        PaymentRequest request,
        CancellationToken cancellationToken);
}
```

You could later have:

EXAMPLE / FLOW

```
public sealed class ProviderAPaymentGateway
    : IPaymentGateway
{
}
```

and:

EXAMPLE / FLOW

```
public sealed class ProviderBPaymentGateway
    : IPaymentGateway
{
}
```

The Application layer depends on:

EXAMPLE / FLOW

```
IPaymentGateway
```

rather than a particular payment provider.

• Polymorphism

Polymorphism means different implementations can be used through the same contract.

Example:

EXAMPLE / FLOW

```
IPaymentGateway
       |
       +---- ProviderAPaymentGateway
       |
       +---- ProviderBPaymentGateway
```

Application code calls:

EXAMPLE / FLOW

```
await paymentGateway.ChargeAsync(...);
```

without needing completely different payment logic for every provider.

• Inheritance

Inheritance means one type derives from another.

Example:

EXAMPLE / FLOW

```
public abstract class DomainException : Exception
{
    protected DomainException(string message)
        : base(message)
    {
    }
}
```

Then:

EXAMPLE / FLOW

```
public sealed class InvalidGiftException
    : DomainException
{
    public InvalidGiftException(string message)
        : base(message)
    {
    }
}
```

Inheritance is useful here because:

EXAMPLE / FLOW

```
InvalidGiftException
IS A
DomainException
```

Do not create deep inheritance trees like:

EXAMPLE / FLOW

```
Entity
  ↓
BusinessEntity
  ↓
FinancialEntity
  ↓
DonationEntity
  ↓
GiftEntity
```

unless they model genuine behavior.

Prefer composition for most domain modeling.

• Composition

Composition means building a larger object from smaller collaborating objects.

Gift:

EXAMPLE / FLOW

```
Gift
 |
 +---- GiftAllocation
 +---- Campaign reference
 +---- Constituent reference
```

This is usually better than trying to make:

EXAMPLE / FLOW

```
Gift inherits Campaign
```

or similar incorrect inheritance.

Easy rule:

EXAMPLE / FLOW

```
"is-a"
→ inheritance may fit

"has-a"
→ composition usually fits
```

SOLID Principles

You should apply SOLID, but not turn the codebase into hundreds of meaningless interfaces.

Single Responsibility Principle

A class should have one main reason to change.

Bad:

EXAMPLE / FLOW

```
GiftService

Creates Gifts
Sends email
Writes SQL
Logs
Processes payments
Generates PDFs
```

Better:

EXAMPLE / FLOW

```
CreateGiftHandler
PaymentGateway
NotificationPublisher
GiftRepository
```

Open/Closed Principle

You should be able to add another payment provider without rewriting all payment workflow code.

Use:

EXAMPLE / FLOW

```
IPaymentGateway
+
provider implementations
```

Liskov Substitution

An implementation of:

EXAMPLE / FLOW

```
IPaymentGateway
```

should behave according to the contract.

A fake gateway for tests should be usable where the real gateway is expected.

Interface Segregation

Do not create:

EXAMPLE / FLOW

```
public interface IEverythingService
{
    Task CreateGift();
    Task ProcessPayment();
    Task SendEmail();
    Task GenerateReport();
    Task RebuildIndex();
}
```

Prefer smaller contracts.

Dependency Inversion

Application should depend on abstractions.

Example:

EXAMPLE / FLOW

```
Application
     |
     v
IPaymentGateway
     ^
     |
Infrastructure implementation
```

not:

EXAMPLE / FLOW

```
Application
     |
     v
ConcreteProviderSdk
```

Records

Records are useful for immutable data-shaped objects.

Good examples:

EXAMPLE / FLOW

```
public sealed record CreateGiftRequest(
    long ConstituentId,
    long? CampaignId,
    decimal TotalAmount,
    string CurrencyCode,
    IReadOnlyList<GiftAllocationRequest> Allocations);
```

DTOs and commands are excellent record candidates.

Entities normally need identity and lifecycle behavior, so classes often fit better.

Generics

Generics allow reusable code while preserving type safety.

Instead of:

EXAMPLE / FLOW

```
object GetSomething();
```

you can have:

EXAMPLE / FLOW

```
Result<T>
```

or:

EXAMPLE / FLOW

```
PagedResult<T>
```

• PagedResult

Very useful in this project:

EXAMPLE / FLOW

```
public sealed record PagedResult<T>(
    IReadOnlyList<T> Items,
    int PageSize,
    string? NextCursor);
```

Then:

EXAMPLE / FLOW

```
PagedResult<ConstituentSummary>
PagedResult<GiftSummary>
PagedResult<PaymentSummary>
```

all reuse the same paging structure.

• Result

A useful pattern for expected application outcomes.

Example:

EXAMPLE / FLOW

```
public sealed record Result<T>(
    bool IsSuccess,
    T? Value,
    Error? Error);
```

Use it for expected business outcomes where throwing an exception would make normal control flow harder to understand.

For example:

EXAMPLE / FLOW

```
Gift allocation doesn't total Gift amount
```

could be returned as a validation/business result.

Unexpected database failure should still behave differently.

• Generic Repository — Be Careful

You may see:

EXAMPLE / FLOW

```
IRepository<T>
```

everywhere in tutorials.

Example:

EXAMPLE / FLOW

```
Add(T entity)
GetById(long id)
Delete(T entity)
```

Do not automatically create a generic repository over EF Core.

Why?

EF Core already provides:

EXAMPLE / FLOW

```
DbSet<T>
DbContext
```

which already resemble repository/unit-of-work abstractions.

A generic repository can hide important features such as:

EXAMPLE / FLOW

```
Include
projection
AsNoTracking
compiled queries
bulk operations
specialized SQL
```

For this project, prefer:

EXAMPLE / FLOW

```
aggregate-specific repositories
```

when the abstraction adds business value.

Example:

EXAMPLE / FLOW

```
public interface IGiftRepository
{
    Task<Gift?> GetForUpdateAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);
}
```

Collections

Collections are essential in this project.

You should understand when each type makes sense.

List

Ordered, index-based collection.

Use for:

EXAMPLE / FLOW

```
Gift allocations
Payment attempts
API DTO lists
```

Example:

EXAMPLE / FLOW

```
List<GiftAllocation>
```

when order matters or sequential access is normal.

IReadOnlyCollection

Expose domain collections without allowing callers to mutate them directly.

Example:

EXAMPLE / FLOW

```
public IReadOnlyCollection<GiftAllocation> Allocations =>
    _allocations.AsReadOnly();
```

This supports encapsulation.

HashSet

Excellent when uniqueness and membership lookup matter.

Example:

While validating Gift allocations:

EXAMPLE / FLOW

```
var fundIds = new HashSet<long>();

foreach (var allocation in request.Allocations)
{
    if (!fundIds.Add(allocation.FundId))
    {
```

```
            throw new DuplicateFundAllocationException(
                allocation.FundId);
        }
    }
```

This detects:

EXAMPLE / FLOW

```
Fund 300
Fund 300
```

appearing twice.

• Dictionary<TKey,TValue>

Use for key-to-value lookup.

Example payment provider factory:

EXAMPLE / FLOW

```
ProviderName
→ IPaymentGateway
```

Conceptually:

EXAMPLE / FLOW

```
Dictionary<string, IPaymentGateway>
```

But if this is shared mutable state in a singleton, thread safety becomes important.

• Queue

First-in-first-out collection.

Useful for learning local queues.

Do not use an in-memory `Queue<T>` as your durable cross-service message system.

If the process crashes:

EXAMPLE / FLOW

```
Queue<T>
```

is lost.

Use later:

EXAMPLE / FLOW

```
Azure Service Bus
```

for durable distributed messaging.

• ConcurrentDictionary<TKey,TValue>

Use when multiple threads genuinely need safe concurrent access to the same in-memory dictionary.

Do not use it as a replacement for:

EXAMPLE / FLOW

```
SQL Server
Redis
distributed cache
```

because every application instance would still have its own memory.

• IEnumerable

Represents a sequence.

Can be:

EXAMPLE / FLOW

```
in-memory
lazy
database-backed indirectly
```

depending on source.

• IQueryable

Represents a query expression that a provider such as EF Core can translate.

Example:

EXAMPLE / FLOW

```
IQueryable<Gift> query = db.Gifts;
```

Then:

EXAMPLE / FLOW

```
query = query.Where(...);
query = query.OrderBy(...);
```

Finally:

EXAMPLE / FLOW

```
await query.ToListAsync();
```

Do not casually expose `IQueryable<T>` all the way through controllers because that leaks persistence behavior throughout your application.

IAsyncEnumerable

Useful for asynchronous streaming.

Potential example:

EXAMPLE / FLOW

```
large reporting export
```

But for normal Angular pages:

EXAMPLE / FLOW

```
pagination
```

is usually better.

LINQ

LINQ allows querying collections and query providers using C#.

Example:

EXAMPLE / FLOW

```
var activeConstituents =
    constituents
        .Where(x => x.Status == ConstituentStatus.Active)
        .OrderBy(x => x.DisplayName)
        .ToList();
```

That is LINQ-to-Objects.

With EF Core:

EXAMPLE / FLOW

```
var result = await db.Constituents
    .Where(x =>
        x.TenantId == tenantId &&
        x.Status == ConstituentStatus.Active)
    .OrderBy(x => x.DisplayName)
    .Select(x => new ConstituentSummary(
        x.ConstituentId,
        x.DisplayName,
        x.PrimaryEmail))
    .ToListAsync(cancellationToken);
```

EF translates suitable LINQ into SQL.

Most Important LINQ Rule

Do as much filtering/projection as appropriate **before materializing**.

Bad:

```
EXAMPLE / FLOW
var all = await db.Constituents.ToListAsync();

var active = all
    .Where(x => x.TenantId == tenantId)
    .ToList();
```

That loads all tenants' rows into memory first.

Better:

```
EXAMPLE / FLOW
var active = await db.Constituents
    .Where(x => x.TenantId == tenantId)
    .ToListAsync();
```

SQL Server performs the filtering.

Select / Projection

For reads, avoid loading complete entities when you only need a small DTO.

Bad:

```
EXAMPLE / FLOW
var gifts = await db.Gifts
    .ToListAsync();
```

then map everything.

Better:

```
EXAMPLE / FLOW
var gifts = await db.Gifts
    .Where(x => x.TenantId == tenantId)
    .Select(x => new GiftSummaryDto(
        x.GiftId,
        x.GiftDate,
        x.TotalAmount,
        x.CurrencyCode))
    .ToListAsync(cancellationToken);
```

Benefits:

```
EXAMPLE / FLOW
fewer columns
less SQL Server I/O
less network data
less EF tracking
less memory
```

AsNoTracking

For read-only EF queries:

EXAMPLE / FLOW

```
db.Gifts.AsNoTracking()
```

avoids normal change-tracking overhead.

Good for:

EXAMPLE / FLOW

```
GET /api/gifts
GET /api/reports/...
GET /api/campaigns/{id}/performance
```

Do not use `AsNoTracking` when you intend to load an entity, modify it, and rely on normal change tracking unless you deliberately attach/update later.

LINQ Operators to Learn in This Project

Use:

EXAMPLE / FLOW

```
Where
Select
OrderBy
ThenBy
GroupBy
Any
All
Contains
FirstOrDefault
SingleOrDefault
Join
GroupJoin
SelectMany
Distinct
Count
Sum
Average
Min
Max
Skip
Take
```

Each has genuine project use.

Any vs Count

Bad existence check:

EXAMPLE / FLOW

```
if (await query.CountAsync() > 0)
```

Better:

EXAMPLE / FLOW

```
if (await query.AnyAsync())
```

because your question is:

EXAMPLE / FLOW

```
Does at least one row exist?
```

not:

EXAMPLE / FLOW

```
Exactly how many rows exist?
```

• SingleOrDefault vs FirstOrDefault

Use `SingleOrDefault` when the business/database rule says:

EXAMPLE / FLOW

```
zero or one
```

and multiple rows indicate a bug.

Use `FirstOrDefault` when:

EXAMPLE / FLOW

```
any first matching row
```

is semantically acceptable.

• Async / Await

This is one of the most important concepts for a high-concurrency Web API.

`async` does not mean:

EXAMPLE / FLOW

```
run everything on another thread.
```

It means code can suspend while waiting for asynchronous work and allow the thread to serve other work.

• Why Async Matters in Web APIs

Suppose 500 requests are waiting for SQL Server.

Synchronous model:

EXAMPLE / FLOW

```
Request
   |
Thread
   |
waiting
waiting
waiting
waiting
```

The thread remains tied up.

Async model:

EXAMPLE / FLOW

```
Request
   |
start SQL I/O
   |
release thread
   |
SQL completes
   |
continuation resumes
```

This improves scalability for I/O-heavy applications.

ASP.NET Core guidance recommends asynchronous hot paths, asynchronous data access, and async controller actions when asynchronous APIs are available. It specifically warns against wrapping synchronous I/O in `Task.Run` merely to make it look asynchronous.

Async Endpoint Example

JSON

```
[HttpGet("{id:long}")]
public async Task<ActionResult<GiftDto>> GetGiftAsync(
    long id,
    CancellationToken cancellationToken)
{
    var result = await _giftQueries.GetByIdAsync(
        id,
        cancellationToken);

    return result is null
        ? NotFound()
        : Ok(result);
}
```

Flow:

EXAMPLE / FLOW

```
Controller async
      |
      v
Application async
      |
      v
EF / Dapper async
      |
      v
SQL Server
```

Keep the entire I/O path asynchronous.

• CancellationToken

Every important endpoint should accept:

EXAMPLE / FLOW

```
CancellationToken cancellationToken
```

and pass it downward.

Example:

EXAMPLE / FLOW

```
Browser cancels request
       |
       v
ASP.NET CancellationToken
       |
       v
Application
       |
       v
EF Core
       |
       v
SQL request may be cancelled
```

Do not accept a CancellationToken in the controller and then ignore it.

• Avoid async void

Bad:

EXAMPLE / FLOW

```
public async void ProcessPayment()
```

Exceptions and completion become difficult to observe.

Prefer:

EXAMPLE / FLOW

```
public async Task ProcessPaymentAsync()
```

`async void` is generally limited to specific event-handler patterns.

Task vs ValueTask

For most application services:

EXAMPLE / FLOW

```
Task<T>
```

is the correct default.

Use:

EXAMPLE / FLOW

```
ValueTask<T>
```

only when profiling and API design show it provides value, usually when operations frequently complete synchronously.

Do not use `ValueTask` everywhere simply because it sounds faster.

Task.WhenAll

Use when you genuinely have **independent asynchronous I/O operations**.

Example Reporting API needs three independent service calls:

EXAMPLE / FLOW

```
Campaign totals
Payment totals
Interaction totals
```

If each is independent:

EXAMPLE / FLOW

```
var campaignsTask =
    _campaignClient.GetSummaryAsync(ct);

var paymentsTask =
    _paymentClient.GetSummaryAsync(ct);

var interactionsTask =
    _interactionClient.GetSummaryAsync(ct);

await Task.WhenAll(
    campaignsTask,
    paymentsTask,
    interactionsTask);
```

Then combine results.

Important EF Core Rule

Do **not** do:

EXAMPLE / FLOW

```
var a = db.Gifts.ToListAsync();
var b = db.Tasks.ToListAsync();

await Task.WhenAll(a, b);
```

on the same `DbContext`.

EF Core does not support multiple parallel operations on one `DbContext`. A `DbContext` is designed as a short-lived unit of work and is not thread-safe.

Correct:

EXAMPLE / FLOW

```
await first operation
then
await second operation
```

or, only if true independent parallel database work is justified:

EXAMPLE / FLOW

```
separate DbContext instances/scopes.
```

But parallel database queries can increase SQL Server load, so do not do this automatically.

Parallelism

Async and parallelism are different.

EXAMPLE / FLOW

```
Async
→ waiting efficiently

Parallelism
→ doing CPU work simultaneously
```

When Parallelism Makes Sense

Example:

You import 50,000 records and must perform expensive independent CPU transformations.

Potentially:

EXAMPLE / FLOW

```
Parallel.ForEach(...)
```

or:

EXAMPLE / FLOW

```
Parallel.ForEachAsync(...)
```

with bounded concurrency.

When Parallelism Does NOT Make Sense

Do not take:

EXAMPLE / FLOW

```
1,000 Gift rows
```

and do:

EXAMPLE / FLOW

```
Parallel.ForEachAsync(
    gifts,
    async gift =>
    {
        await db.SaveChangesAsync();
    });
```

using the same DbContext.

That is incorrect.

Also, running 500 parallel SQL requests can make database performance much worse.

Async Rule for This Project

Use async by default for:

EXAMPLE / FLOW

```
EF Core queries

SaveChangesAsync

Dapper ExecuteAsync/QueryAsync

HttpClient

Service Bus

Blob Storage

Key Vault

file I/O

external payment providers
```

Use parallelism only after asking:

EXAMPLE / FLOW

```
Are these operations independent?

Is this CPU-bound?

Is shared state thread-safe?

Will this overload SQL Server or another dependency?

Is concurrency bounded?
```

• Dependency Injection

Dependency Injection means a class receives the services it needs instead of creating them itself.

Bad:

EXAMPLE / FLOW

```
public sealed class GiftService
{
    private readonly SqlConnection _connection =
        new(...);

    private readonly PaymentProviderClient _client =
        new(...);
}
```

The class controls concrete dependencies.

Hard to:

```
EXAMPLE / FLOW
test
replace
configure
mock/fake
```

Better:

```
EXAMPLE / FLOW
public sealed class CreateGiftHandler
{
    private readonly IGiftRepository _gifts;
    private readonly IUnitOfWork _unitOfWork;

    public CreateGiftHandler(
        IGiftRepository gifts,
        IUnitOfWork unitOfWork)
    {
        _gifts = gifts;
        _unitOfWork = unitOfWork;
    }
}
```

ASP.NET Core has a built-in DI container. Microsoft recommends constructor injection and small, well-factored services rather than stateful static access patterns.

• Transient

Registration:

```
EXAMPLE / FLOW
services.AddTransient<IExample, Example>();
```

Meaning:

```
EXAMPLE / FLOW
Every resolution
→ new instance
```

Good for:

```
EXAMPLE / FLOW
small stateless services
formatters
mappers
simple validators
```

Example:

```
EXAMPLE / FLOW
services.AddTransient<IGiftRequestValidator,
                      GiftRequestValidator>();
```

• Scoped

Registration:

EXAMPLE / FLOW

```
services.AddScoped<IExample, Example>();
```

Meaning in ASP.NET Core:

EXAMPLE / FLOW

```
one instance
per HTTP request scope
```

This is the most important lifetime for database work.

EF Core's `AddDbContext` uses scoped lifetime by default because a DbContext normally represents a request-sized unit of work.

Use scoped for:

EXAMPLE / FLOW

```
FundraisingDbContext

PaymentDbContext

TenantContext

many application handlers/services
that coordinate one request
```

• Singleton

Registration:

EXAMPLE / FLOW

```
services.AddSingleton<IExample, Example>();
```

Meaning:

EXAMPLE / FLOW

```
one application instance
for process lifetime
```

It must be:

EXAMPLE / FLOW

```
thread-safe
```

Good examples:

EXAMPLE / FLOW

```
stateless expensive shared service
TimeProvider
immutable metadata
thread-safe cache abstraction
```

Be extremely careful with mutable state.

Never inject:

EXAMPLE / FLOW

```
scoped DbContext
```

into a singleton.

Microsoft's DI guidance specifically warns that singletons must be thread-safe and should not capture scoped services.

• Scoped Example — Tenant Context

EXAMPLE / FLOW

```
public interface ITenantContext
{
    long TenantId { get; }
}
```

Each HTTP request represents one authenticated tenant context.

Therefore:

EXAMPLE / FLOW

```
Scoped
```

fits naturally.

• Singleton Example — TimeProvider

Instead of calling:

EXAMPLE / FLOW

```
DateTime.UtcNow
```

everywhere, use:

EXAMPLE / FLOW

```
TimeProvider
```

which makes time-related code easier to test.

Register a shared instance.

DI Lifetime Memory Rule

EXAMPLE / FLOW

```
Transient
→ new whenever resolved

Scoped
→ one per HTTP request

Singleton
→ one for application lifetime
```

Static Classes

`static` means the member/type belongs to the type rather than an object instance.

Good uses:

EXAMPLE / FLOW

```
constants

pure utility methods

extension methods

registration extensions
```

Example:

EXAMPLE / FLOW

```
public static class ServiceCollectionExtensions
{
    public static IServiceCollection AddFundraisingApplication(
        this IServiceCollection services)
    {
        // registrations

        return services;
    }
}
```

This is a good static class because it has:

EXAMPLE / FLOW

```
no per-user mutable state.
```

Bad Static State

Do not create:

EXAMPLE / FLOW

```
public static class CurrentTenant
{
    public static long TenantId { get; set; }
}
```

With 1,000 concurrent users:

EXAMPLE / FLOW

```
User A sets TenantId = 1
User B sets TenantId = 2
```

Now global shared state becomes corrupted.

Use:

EXAMPLE / FLOW

```
Scoped ITenantContext
```

instead.

• Static vs Singleton

They are not identical.

Static:

EXAMPLE / FLOW

```
cannot normally implement interface

cannot be naturally injected

global type-level state

harder to replace during tests
```

Singleton:

EXAMPLE / FLOW

```
normal object

can implement interface

can use DI

can be replaced in tests

lifetime controlled by container
```

Prefer DI singleton when you have a service abstraction.

Use static mainly for:

EXAMPLE / FLOW

```
pure helpers
constants
extension methods
```

Program.cs and Startup

Older ASP.NET Core applications commonly used:

EXAMPLE / FLOW

```
Startup.cs

ConfigureServices()

Configure()
```

Modern .NET uses the minimal hosting model with:

EXAMPLE / FLOW

```
Program.cs
```

by default.

Do not let Program.cs become 1,000 lines.

Instead:

EXAMPLE / FLOW

```
var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddApiServices(builder.Configuration)
    .AddApplication()
    .AddInfrastructure(builder.Configuration);

var app = builder.Build();

app.UseApplicationPipeline();

app.Run();
```

Then organize registrations into extension methods.

Example Service Registration

EXAMPLE / FLOW

```
public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
```

```
        services.AddDbContext<FundraisingDbContext>(
            options =>
                options.UseSqlServer(
                    configuration.GetConnectionString(
                        "FundraisingDb")));

        return services;
    }
}
```

Program.cs remains small.

Configuration

ASP.NET Core configuration can come from:

EXAMPLE / FLOW

```
appsettings.json

appsettings.Development.json

environment variables

command-line arguments

user secrets locally

Azure Key Vault later
```

Never commit:

EXAMPLE / FLOW

```
production passwords
API secrets
payment provider private keys
```

to Git.

appsettings.json

Example:

JSON

```
{
  "ConnectionStrings": {
    "FundraisingDb": ""
  },

  "PaymentProvider": {
    "BaseUrl": "https://provider.example",
    "TimeoutSeconds": 30
  }
}
```

Do not store actual production secrets in committed configuration.

Options Pattern

Instead of:

EXAMPLE / FLOW

```
configuration["PaymentProvider:TimeoutSeconds"]
```

throughout the project, define:

EXAMPLE / FLOW

```
public sealed class PaymentProviderOptions
{
    public const string SectionName = "PaymentProvider";

    public required string BaseUrl { get; init; }

    public int TimeoutSeconds { get; init; }
}
```

Register:

EXAMPLE / FLOW

```
services
    .AddOptions<PaymentProviderOptions>()
    .BindConfiguration(
        PaymentProviderOptions.SectionName)
    .Validate(
        x => x.TimeoutSeconds > 0,
        "Timeout must be greater than zero.")
    .ValidateOnStart();
```

The Options pattern provides strongly typed configuration, separation of concerns, and validation. Current ASP.NET Core supports `IOptions<T>`, `IOptionsSnapshot<T>`, `IOptionsMonitor<T>`, and startup validation.

IOptions

Good when configuration is effectively stable for the process.

IOptionsSnapshot

Scoped.

Can recompute configuration for each request when supported by the provider.

Do not use it automatically everywhere because it has additional per-request work.

IOptionsMonitor

Singleton-compatible.

Useful for:

EXAMPLE / FLOW

```
configuration change notifications

current configuration value

reloadable options
```

Clean Architecture Command Example

Create Gift command:

EXAMPLE / FLOW

```
public sealed record CreateGiftCommand(
    long ConstituentId,
    long? CampaignId,
    DateOnly GiftDate,
    decimal TotalAmount,
    string CurrencyCode,
    IReadOnlyList<CreateGiftAllocation> Allocations);
```

Handler:

EXAMPLE / FLOW

```
public sealed class CreateGiftHandler
{
    private readonly IGiftWriter _giftWriter;
    private readonly ITenantContext _tenantContext;

    public CreateGiftHandler(
        IGiftWriter giftWriter,
        ITenantContext tenantContext)
    {
        _giftWriter = giftWriter;
        _tenantContext = tenantContext;
    }

    public async Task<CreateGiftResult> HandleAsync(
        CreateGiftCommand command,
        CancellationToken cancellationToken)
    {
        return await _giftWriter.CreateAsync(
            _tenantContext.TenantId,
            command,
            cancellationToken);
    }
}
```

Infrastructure:

EXAMPLE / FLOW

```
IGiftWriter
      |
      v
SqlGiftWriter
```

which calls:

EXAMPLE / FLOW

```
fundraising.usp_CreateGift
```

or performs EF Core transaction logic.

• CQRS

CQRS means:

EXAMPLE / FLOW

```
Commands
→ change state

Queries
→ read state
```

It does **not** automatically mean:

EXAMPLE / FLOW

```
separate database
event sourcing
Kafka
microservices
```

You can use simple CQRS in one API.

Example:

EXAMPLE / FLOW

```
CreateGiftCommand
UpdateGiftCommand
CompleteTaskCommand

GetGiftQuery
SearchGiftsQuery
GetConstituentSummaryQuery
```

Why useful?

Write workflows and read workflows often have very different needs.

Example:

EXAMPLE / FLOW

```
Create Gift
→ transaction
→ validation
→ Outbox

Get Gift
→ projection
→ AsNoTracking
```

Keeping these separate improves clarity.

• Mediator Pattern

A mediator lets controllers send requests without directly knowing every handler dependency.

Conceptually:

EXAMPLE / FLOW

```
Controller
   |
   v
Mediator
   |
   v
Handler
```

You can use a library or implement a simple dispatcher.

Do not add a mediator library simply because every Clean Architecture tutorial has one.

Use it if it meaningfully reduces coupling between:

EXAMPLE / FLOW

```
API
and
Application handlers.
```

• Controllers

Controller example:

JSON

```
[ApiController]
[Route("api/v{version:apiVersion}/gifts")]
public sealed class GiftsController : ControllerBase
{
    private readonly CreateGiftHandler _createGift;

    public GiftsController(
        CreateGiftHandler createGift)
    {
        _createGift = createGift;
    }
```

```
    [HttpPost]
    public async Task<ActionResult<CreateGiftResponse>>
        CreateAsync(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        // map request -> command

        // execute handler

        // return HTTP result
    }
}
```

Controllers should mostly contain:

EXAMPLE / FLOW

```
HTTP concerns
mapping
dispatching
response conversion
```

not core business rules.

HTTP Methods

Use normal HTTP semantics.

EXAMPLE / FLOW

```
GET
→ read

POST
→ create/start action

PUT
→ replace/update a known resource

PATCH
→ partial modification

DELETE
→ delete/cancel where appropriate
```

Status Codes

Use meaningful HTTP responses.

EXAMPLE / FLOW

```
200 OK
→ normal successful read/update

201 Created
```

```
→ resource created

204 No Content
→ successful command with no response body

400 Bad Request
→ malformed/invalid request

401 Unauthorized
→ not authenticated

403 Forbidden
→ authenticated but not permitted

404 Not Found
→ resource absent

409 Conflict
→ concurrency/business conflict

412 Precondition Failed
→ useful if using HTTP preconditions/ETag

422 Unprocessable Content
→ optional choice for semantic validation

429 Too Many Requests
→ rate limiting

500 Internal Server Error
→ unexpected server error
```

Be consistent across all APIs.

• DTOs

Do not expose EF entities directly from controllers.

Bad:

EXAMPLE / FLOW

```
return Ok(db.Constituents);
```

That couples:

EXAMPLE / FLOW

```
database entity
API contract
```

together.

Use DTOs.

Example:

EXAMPLE / FLOW
```
public sealed record ConstituentResponse(
    long Id,
    string DisplayName,
    string? PrimaryEmail,
    string? PrimaryPhone,
    byte Status);
```

Request DTO

EXAMPLE / FLOW
```
public sealed record CreateConstituentRequest(
    byte ConstituentType,
    string DisplayName,
    string? FirstName,
    string? LastName,
    string? PrimaryEmail,
    string? PrimaryPhone);
```

Mapping

For learning, write some mappings manually.

Why?

You should understand:

EXAMPLE / FLOW
```
API DTO
↔
Application command
↔
Domain
```

before hiding mapping behind a library.

Example:

EXAMPLE / FLOW
```
var command = new CreateConstituentCommand(
    request.ConstituentType,
    request.DisplayName,
    request.FirstName,
    request.LastName,
    request.PrimaryEmail,
    request.PrimaryPhone);
```

Later a mapper library can be introduced where it genuinely removes repetitive code.

Validation

Validation exists at several levels.

EXAMPLE / FLOW

```
Angular validation

API request validation

Application/business validation

Domain invariants

SQL constraints
```

These layers are complementary.

Request Validation

Example:

EXAMPLE / FLOW

```
DisplayName
→ required

Email
→ correct basic format

PageSize
→ 1..200
```

This should fail before expensive business work.

Business Validation

Example:

EXAMPLE / FLOW

```
Gift allocation total
=
Gift total
```

belongs deeper than controller validation.

Database Validation

SQL still has:

EXAMPLE / FLOW

```
CHECK TotalAmount > 0
```

because application validation cannot be the database's only protection.

API Versioning

API versioning protects clients when API contracts evolve.

Suppose v1 returns:

JSON

```
{
  "giftId": 10000,
  "amount": 1000
}
```

Later you want a breaking contract:

JSON

```
{
  "id": 10000,
  "money": {
    "amount": 1000,
    "currency": "USD"
  }
}
```

Changing v1 directly may break Angular/mobile/external clients.

Instead:

EXAMPLE / FLOW

```
v1
continues old contract

v2
introduces breaking contract
```

Versioning Strategy for This Project

Use URL segment versioning initially because it is clear while learning:

EXAMPLE / FLOW

```
/api/v1/constituents

/api/v1/gifts

/api/v1/payments
```

later:

EXAMPLE / FLOW

```
/api/v2/gifts
```

ASP.NET API Versioning currently supports controllers and Minimal APIs through the `Asp.Versioning.*` packages; the newer packages use names such as `Asp.Versioning.Mvc` and API Explorer support rather than the old `Microsoft.AspNetCore.Mvc.Versioning` naming.

API Versioning Configuration Concept

EXAMPLE / FLOW

```
services
    .AddApiVersioning(options =>
    {
        options.ReportApiVersions = true;
    })
    .AddMvc()
    .AddApiExplorer(options =>
    {
        options.GroupNameFormat = "'v'VVV";
    });
```

Then controller:

JSON

```
[ApiVersion(1.0)]
[ApiController]
[Route("api/v{version:apiVersion}/gifts")]
public sealed class GiftsController : ControllerBase
{
}
```

Do not create:

EXAMPLE / FLOW

```
v2
v3
v4
```

every time an internal implementation changes.

Version only when the **external API contract has a breaking change**.

API Deprecation

Eventually:

EXAMPLE / FLOW

```
v1
→ supported but deprecated

v2
→ preferred

later
v1 removed after communicated policy
```

Versioning libraries can report supported/deprecated versions to clients.

• OpenAPI / Swagger

OpenAPI describes your HTTP contract.

It documents:

EXAMPLE / FLOW

```
routes

methods

parameters

request bodies

responses

authentication schemes

schemas
```

ASP.NET Core 10 supports OpenAPI 3.1 generation.

Use an interactive UI such as:

EXAMPLE / FLOW

```
Swagger UI
or
Scalar
```

during development.

• OpenAPI Should Show

For:

EXAMPLE / FLOW

```
POST /api/v1/gifts
```

document:

EXAMPLE / FLOW

```
Request body

201 response

400 validation

401 authentication

403 authorization

404 constituent/campaign/fund

409 concurrency/business conflict

500 unexpected failure
```

This makes the API much easier for Angular and future service consumers to use.

Exception Handling

Do not wrap every controller in:

EXAMPLE / FLOW

```
try
{
}
catch
{
}
```

Create centralized handling.

Current ASP.NET Core supports exception-handler middleware and standardized `ProblemDetails` responses for API errors.

ProblemDetails

Example API error:

JSON

```
{
  "type": "https://errors.example/gift-allocation",
  "title": "Gift allocation is invalid.",
  "status": 400,
  "detail": "Allocations total 900 but the gift total is 1000.",
  "traceId": "..."
}
```

Clients receive a predictable error shape.

Custom Exceptions

Examples:

EXAMPLE / FLOW

```
public sealed class EntityNotFoundException : Exception
{
}
```

EXAMPLE / FLOW

```
public sealed class ConcurrencyConflictException
    : Exception
{
}
```

EXAMPLE / FLOW

```
public sealed class DuplicateIdempotencyKeyException
    : Exception
{
}
```

EXAMPLE / FLOW

```
public sealed class PaymentProviderException
    : Exception
{
}
```

Use custom exception types when the type itself helps the application determine handling.

Exception Hierarchy

You could have:

EXAMPLE / FLOW

```
Exception
   |
   +---- DomainException
   |       |
   |       +---- InvalidGiftException
   |       +---- InvalidRefundException
   |
   +---- ApplicationExceptionBase
           |
           +---- EntityNotFoundException
           +---- ConcurrencyConflictException
```

Do not create 100 exception types without meaningful differences.

Generic Exceptions?

You technically can build:

EXAMPLE / FLOW

```
EntityNotFoundException<TEntity>
```

but do not create generic exceptions merely to use generics.

Often this is simpler:

EXAMPLE / FLOW

```
throw new EntityNotFoundException(
    "Gift",
    giftId);
```

Generics should improve type safety or reuse—not increase complexity.

Global Exception Handler

Modern design:

EXAMPLE / FLOW

```
Exception
    |
    v
Global exception handler
    |
    v
Map exception type
    |
    v
ProblemDetails
    |
    v
HTTP status
```

Example mappings:

EXAMPLE / FLOW

```
Validation exception
→ 400

EntityNotFoundException
→ 404

ConcurrencyConflictException
→ 409

UnauthorizedAccessException
→ 403/401 depending context

unexpected exception
→ 500
```

Never return raw:

EXAMPLE / FLOW

```
SQL exception text
stack trace
connection string
provider secret
```

to the client.

Log details internally.

Result Pattern vs Exceptions

Use Results for expected outcomes.

Example:

EXAMPLE / FLOW

```
Gift allocations don't equal Gift total
```

may be an expected business validation failure.

Use exceptions for:

EXAMPLE / FLOW

```
unexpected infrastructure failure
broken invariant
database unavailable
external provider failure
```

The exact balance is architectural, but do not use exceptions like `goto` for every normal branch.

EF Core

Use EF Core primarily for:

EXAMPLE / FLOW

```
normal CRUD

aggregate updates

simple/medium queries

change tracking

migrations where desired
```

Use:

EXAMPLE / FLOW

```
FundraisingDbContext
PaymentDbContext
ReportingDbContext
```

separately.

Never create:

EXAMPLE / FLOW

```
EverythingDbContext
```

across all three databases.

• FundraisingDbContext

Example:

EXAMPLE / FLOW

```
public sealed class FundraisingDbContext
    : DbContext
{
    public DbSet<Constituent> Constituents => Set<Constituent>();

    public DbSet<Campaign> Campaigns => Set<Campaign>();

    public DbSet<Gift> Gifts => Set<Gift>();

    public DbSet<GiftAllocation> GiftAllocations =>
        Set<GiftAllocation>();
}
```

Keep mappings in configuration classes:

EXAMPLE / FLOW

```
Persistence/
  Configurations/
     GiftConfiguration.cs
     ConstituentConfiguration.cs
```

rather than a giant `OnModelCreating`.

• Fluent Configuration

Example:

EXAMPLE / FLOW

```
public sealed class GiftConfiguration
    : IEntityTypeConfiguration<Gift>
{
    public void Configure(
        EntityTypeBuilder<Gift> builder)
    {
        builder.ToTable("Gift", "fundraising");

        builder.HasKey(x => x.Id);

        builder.Property(x => x.TotalAmount)
```

```
            .HasPrecision(19, 4);

        builder.Property(x => x.RowVersion)
            .IsRowVersion();
    }
}
```

RowVersion in EF Core

Configure:

EXAMPLE / FLOW

```
builder.Property(x => x.RowVersion)
    .IsRowVersion();
```

When the row changed since it was loaded:

EXAMPLE / FLOW

```
SaveChangesAsync
```

can throw:

EXAMPLE / FLOW

```
DbUpdateConcurrencyException.
```

Current EF Core specifically documents this concurrency behavior for unexpected affected-row counts.

Map that to:

EXAMPLE / FLOW

```
409 Conflict
```

or, if using ETag/precondition semantics:

EXAMPLE / FLOW

```
412 Precondition Failed.
```

EF Core Tracking

Tracked query:

EXAMPLE / FLOW

```
Get Gift
modify Gift
SaveChanges
```

Read-only query:

```
EXAMPLE / FLOW
AsNoTracking
projection
```

Use the appropriate mode.

• Avoid N+1 Queries

Bad:

```
EXAMPLE / FLOW
load 100 Constituents
```

then:

```
EXAMPLE / FLOW
for each Constituent
load Gifts separately
```

Result:

```
EXAMPLE / FLOW
1 Constituent query
+
100 Gift queries
```

This is the N+1 problem.

Instead:

```
EXAMPLE / FLOW
projection
join
aggregate
appropriate Include
or specialized query
```

depending on requirement.

• Dapper

Dapper is useful when you want:

EXAMPLE / FLOW

```
direct SQL

stored procedures

high-control projections

reporting queries

complex SQL already tuned in SQL Server
```

Excellent candidates:

EXAMPLE / FLOW

```
Constituent timeline

Campaign performance

Executive reports

Stored procedure calls

ReportingDb queries
```

• EF Core vs Dapper

Use both.

EXAMPLE / FLOW

```
EF Core
→ domain CRUD and unit-of-work behavior

Dapper
→ specialized reads / stored procedures
```

Do not turn this into:

EXAMPLE / FLOW

```
EF is bad
Dapper is good
```

or the opposite.

Choose per use case.

• Stored Procedures from .NET

Your SQL document created:

EXAMPLE / FLOW

```
fundraising.usp_CreateGift
```

.NET should call it asynchronously and parameterize values.

The Application layer should not know low-level SqlParameter details.

Example:

EXAMPLE / FLOW

```
Application
    |
    v
IGiftWriter
    |
    v
Infrastructure
    |
    v
Dapper / SqlCommand
    |
    v
usp_CreateGift
```

• Transaction Strategy

There are two main cases.

EF-managed unit of work

If all database modifications use one DbContext:

EXAMPLE / FLOW

```
await db.SaveChangesAsync(cancellationToken);
```

often provides the required local transactional behavior.

• Explicit Transaction

For multiple important operations:

EXAMPLE / FLOW

```
Gift
Allocations
Outbox
```

use:

EXAMPLE / FLOW

```
stored procedure transaction
```

or:

EXAMPLE / FLOW

```
await using var transaction =
    await db.Database.BeginTransactionAsync(ct);
```

Then:

EXAMPLE / FLOW

```
perform work
SaveChanges
Commit
```

• Do Not Create Distributed Transactions

Do not do:

EXAMPLE / FLOW

```
FundraisingDb transaction
+
PaymentDb transaction
+
HTTP payment provider
```

as one giant transaction.

Use:

EXAMPLE / FLOW

```
local transaction
+
Outbox
+
event
+
idempotent consumer
```

• Repository Pattern

Use only where it creates a useful abstraction around an aggregate.

Example:

EXAMPLE / FLOW

```
public interface IConstituentRepository
{
    Task<Constituent?> GetAsync(
        long tenantId,
        long constituentId,
        CancellationToken cancellationToken);

    void Add(Constituent constituent);
}
```

Avoid repositories like:

EXAMPLE / FLOW

```
GetAll()
Update()
Delete()
Find(Expression<Func<T,bool>>)
```

for every entity if they only duplicate EF Core.

Unit of Work Pattern

EF `DbContext` already behaves like a Unit of Work.

You may expose:

EXAMPLE / FLOW

```
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(
        CancellationToken cancellationToken);
}
```

implemented by your DbContext.

But don't wrap EF Core with multiple redundant abstraction layers merely for pattern names.

Factory Pattern

Excellent for payment providers.

EXAMPLE / FLOW

```
PaymentGatewayFactory
      |
      +---- Provider A
      +---- Provider B
```

Example:

EXAMPLE / FLOW

```
public interface IPaymentGatewayFactory
{
    IPaymentGateway Create(string providerName);
}
```

This lets configuration select provider dynamically.

Strategy Pattern

Useful when the algorithm changes.

Example:

EXAMPLE / FLOW

```
Fraud strategy

Payment processing strategy

Refund strategy
```

Conceptually:

EXAMPLE / FLOW

```
IPaymentProcessingStrategy
       |
       +---- CardPaymentStrategy
       +---- WalletPaymentStrategy
       +---- DirectDebitStrategy
```

Use only if these modes truly have different behaviors.

• Adapter Pattern

Excellent for external provider SDKs.

Suppose provider SDK has:

EXAMPLE / FLOW

```
ProviderSdk.CreateCharge(...)
```

Your application wants:

EXAMPLE / FLOW

```
IPaymentGateway.ChargeAsync(...)
```

The Infrastructure adapter translates between them.

EXAMPLE / FLOW

```
Application contract
       |
       v
ProviderAdapter
       |
       v
Third-party SDK
```

• Decorator Pattern

A decorator wraps another implementation to add behavior.

Example:

EXAMPLE / FLOW

```
PaymentGateway
```

wrapped by:

EXAMPLE / FLOW

```
LoggingPaymentGateway
```

or:

EXAMPLE / FLOW

```
MetricsPaymentGateway
```

Flow:

EXAMPLE / FLOW

```
Application
    |
    v
Metrics decorator
    |
    v
Logging decorator
    |
    v
Actual Payment Gateway
```

Useful for cross-cutting concerns without polluting the core implementation.

• Specification Pattern

Could represent reusable query rules.

Example:

EXAMPLE / FLOW

```
Active Constituents

Overdue Tasks

High-Value Gifts
```

Do not add Specification pattern if normal application query objects are already clear.

• Outbox Pattern

Already designed in SQL.

.NET responsibility:

EXAMPLE / FLOW

```
Business command
      |
      v
database transaction
      |
      +---- business row
      +---- OutboxMessage
      |
      v
commit
```

Then:

EXAMPLE / FLOW

```
BackgroundService
      |
      v
read unprocessed Outbox
      |
      v
publish Service Bus
      |
      v
mark processed
```

• BackgroundService

ASP.NET Core background workers can implement:

EXAMPLE / FLOW

```
BackgroundService
```

Example:

EXAMPLE / FLOW

```
OutboxPublisher
```

Remember:

A hosted background service is effectively singleton-like.

If it needs:

EXAMPLE / FLOW

```
scoped DbContext
```

create a scope per processing cycle with:

EXAMPLE / FLOW

```
IServiceScopeFactory
```

rather than injecting the scoped DbContext directly into the singleton worker. Microsoft DI guidance specifically recommends creating a scope when singleton/background code must use scoped services.

Outbox Worker Flow

EXAMPLE / FLOW

```
Background worker
      |
      v
Create DI scope
      |
      v
Resolve FundraisingDbContext
      |
      v
Read 100 unprocessed messages
      |
      v
Publish
      |
      v
Mark successfully processed
      |
      v
Dispose scope
```

Process in controlled batches.

Do not load:

EXAMPLE / FLOW

```
5 million Outbox rows
```

into memory.

Idempotency Pattern in .NET

Payment endpoint:

EXAMPLE / FLOW

```
POST /api/v1/payments
```

requires:

EXAMPLE / FLOW

```
Idempotency-Key
```

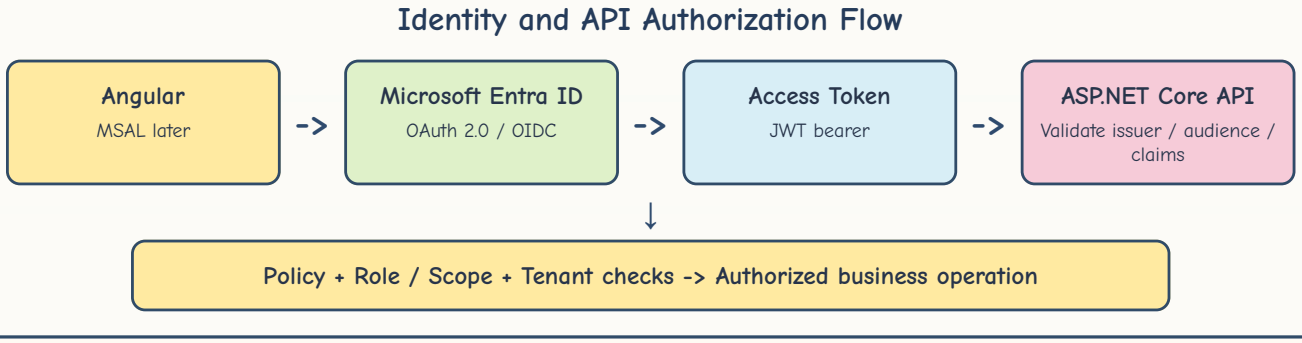
Flow:

EXAMPLE / FLOW

```
Controller
   |
   v
CreatePaymentCommand
   |
   v
Idempotency service
   |
   +---- already completed?
   |        |
   |        v
   |   return prior result
   |
   v
Create Payment
   |
   v
Store response metadata
```

SQL unique constraint remains the final race-condition protection.

Authentication



Authentication answers:

> Who is making this request?

Your platform will use:

EXAMPLE / FLOW

```
Microsoft Entra ID
```

as the main identity provider.

Do not build your own username/password/token system for this project.

Authorization

Authorization answers:

> What is this authenticated identity allowed to do?

Example:

EXAMPLE / FLOW

```
Fundraiser
→ Create Gift

Finance User
→ Request Refund

Reporting User
→ View reports

Tenant Administrator
→ Manage application users
```

• OAuth 2.0

OAuth 2.0 is mainly an authorization framework.

For your Angular + Web API architecture:

EXAMPLE / FLOW

```
Angular
   |
   | OAuth authorization flow
   v
Microsoft Entra ID
   |
   | access token
   v
Angular
   |
   | Authorization: Bearer <token>
   v
ASP.NET Core API
```

The API validates the token.

• OpenID Connect

OpenID Connect adds authentication/identity semantics on top of OAuth 2.0.

Easy memory rule:

EXAMPLE / FLOW

```
OAuth 2.0
→ authorization / API access

OpenID Connect
→ identity / sign-in
```

Modern ASP.NET Core supports OIDC authentication, and Microsoft's current guidance for interactive web clients recommends authorization-code-based flows and PKCE where applicable.

• Access Token vs ID Token

Do not confuse these.

EXAMPLE / FLOW

```
ID token
→ tells client about authenticated identity

Access token
→ presented to API to authorize API access
```

Angular should call your API using:

EXAMPLE / FLOW

```
access token
```

not send the ID token as the API authorization credential.

• JWT Bearer Authentication

Your ASP.NET Core API will usually receive:

EXAMPLE / FLOW

```
Authorization:
Bearer <access-token>
```

Configure:

EXAMPLE / FLOW

```
JWT bearer authentication
```

The API validates:

EXAMPLE / FLOW

```
signature

issuer

audience

expiration

claims
```

Microsoft recommends standards-based access tokens rather than inventing your own token format.

Angular + MSAL Relationship

Later:

EXAMPLE / FLOW

```
Angular
   |
   v
MSAL
   |
   v
Microsoft Entra ID
   |
   v
Access Token
   |
   v
ASP.NET Core API
```

The .NET API does not use Angular's authentication state as proof.

It validates the access token itself.

SAML

SAML is another identity federation/SSO standard, especially common in enterprise/legacy environments.

Do not use SAML directly as the normal API authorization mechanism.

Possible architecture:

EXAMPLE / FLOW

```
Corporate identity provider
       |
       | SAML federation
       v
Microsoft Entra ID
       |
       | OAuth/OIDC
       v
Angular + APIs
```

So you should learn SAML concepts because enterprise authentication uses them, but the normal Angular-to-Web-API request will use:

EXAMPLE / FLOW

```
OAuth/OIDC access tokens
```

rather than sending a SAML assertion to every API.

Claims

Claims are pieces of identity/security information.

Examples:

EXAMPLE / FLOW

```
user object ID

tenant ID

roles

scopes

email

name
```

Do not trust arbitrary client request fields such as:

JSON

```
{
  "tenantId": 999
}
```

for security.

Determine tenant context from trusted authenticated claims and authorized server-side mappings.

• Roles

Possible application roles:

EXAMPLE / FLOW

```
TenantAdministrator

FundraisingManager

Fundraiser

FinanceUser

PaymentOperations

ReportingUser
```

• Policy-Based Authorization

Prefer business-oriented policies.

Example:

EXAMPLE / FLOW

```
services.AddAuthorization(options =>
{
    options.AddPolicy(
        "CanRefundPayment",
```

```
        policy =>
        {
            policy.RequireAuthenticatedUser();
            policy.RequireRole(
                "FinanceUser",
                "PaymentOperations");
        });
});
```

Controller:

JSON

```
[Authorize(Policy = "CanRefundPayment")]
[HttpPost("{paymentId:long}/refunds")]
```

• Scope-Based Authorization

For service/API authorization you may have scopes such as:

EXAMPLE / FLOW

```
fundraising.read
fundraising.write

payments.read
payments.write

reporting.read
```

Scopes answer:

EXAMPLE / FLOW

```
What API capability has this caller been granted?
```

Roles often answer:

EXAMPLE / FLOW

```
What organizational role does this person have?
```

You may use both.

• Tenant Authorization

Authentication is not enough.

Suppose Sarah is:

EXAMPLE / FLOW

```
TenantId = 1
```

Request:

EXAMPLE / FLOW

```
GET /api/v1/constituents/500
```

You must ensure Constituent 500 belongs to:

EXAMPLE / FLOW

```
TenantId = 1.
```

Do not merely query:

EXAMPLE / FLOW

```
db.Constituents
  .SingleAsync(x => x.Id == id);
```

Use:

EXAMPLE / FLOW

```
db.Constituents
  .SingleOrDefaultAsync(x =>
      x.Id == id &&
      x.TenantId == tenantContext.TenantId);
```

SQL RLS later provides another protection layer.

• Middleware Pipeline

Typical conceptual pipeline:

EXAMPLE / FLOW

```
Request
   |
   v
Forwarded headers when required
   |
   v
HTTPS
   |
   v
Exception handling
   |
   v
Routing
   |
   v
CORS
   |
   v
Authentication
   |
   v
Authorization
   |
   v
```

```
Rate limiting
   |
   v
Endpoint / Controller
   |
   v
Response
```

Exact middleware order matters.

Do not randomly rearrange middleware.

CORS

CORS controls which browser origins may call your API.

Example:

EXAMPLE / FLOW

```
Angular
https://app.example.com
```

allowed to call:

EXAMPLE / FLOW

```
https://api.example.com
```

Do not configure production CORS as:

EXAMPLE / FLOW

```
AllowAnyOrigin
AllowAnyMethod
AllowAnyHeader
```

without understanding the security implications.

Use explicit origins.

Rate Limiting

Rate limiting protects endpoints from excessive traffic.

Good candidates:

EXAMPLE / FLOW

```
search

payment creation

webhook abuse protection

expensive reports
```

Policies might distinguish:

EXAMPLE / FLOW

```
normal authenticated users

administrators

payment-provider webhooks
```

Do not rely on rate limiting as authentication.

HttpClient

For external APIs:

EXAMPLE / FLOW

```
Payment Provider
Microsoft Graph
future AI APIs
```

use:

EXAMPLE / FLOW

```
HttpClientFactory
```

and typed clients.

Example:

EXAMPLE / FLOW

```
public sealed class ProviderAPaymentClient
{
    private readonly HttpClient _httpClient;

    public ProviderAPaymentClient(
        HttpClient httpClient)
    {
        _httpClient = httpClient;
    }
}
```

Registration:

EXAMPLE / FLOW

```
AddHttpClient<ProviderAPaymentClient>()
```

Resilience

External services fail.

Possible failures:

EXAMPLE / FLOW

```
timeout

503

connection failure

rate limit
```

Use resilience patterns:

EXAMPLE / FLOW

```
timeout

retry

circuit breaker
```

carefully.

Do not blindly retry:

EXAMPLE / FLOW

```
charge credit card
```

unless the provider operation is idempotent or protected with a provider idempotency key.

Otherwise:

EXAMPLE / FLOW

```
retry
→ duplicate charge
```

may occur.

• Retry Pattern

Good candidate:

EXAMPLE / FLOW

```
GET provider status
```

Transient failure:

EXAMPLE / FLOW

```
503
```

retry may be reasonable.

Circuit Breaker

If provider repeatedly fails:

EXAMPLE / FLOW

```
Request
Request
Request
Request
```

do not continuously hammer it.

Circuit breaker temporarily opens:

EXAMPLE / FLOW

```
Provider unhealthy
     |
     v
fail quickly / degrade
```

then later probes for recovery.

Timeout

Every external call should have sensible timeout behavior.

Never let a payment-provider call hang indefinitely.

Caching

Use caching where the data is:

EXAMPLE / FLOW

```
read frequently

expensive to produce

safe to reuse temporarily
```

Possible examples:

EXAMPLE / FLOW

```
public campaign catalog

fund metadata

executive report summaries
```

Do not cache:

```
EXAMPLE / FLOW
payment state
```

for long periods when users need current financial state.

Output Cache

ASP.NET Core has output caching middleware for controller and Minimal API endpoints. It supports policies and can later use Redis-backed storage when multiple server instances need shared cache.

Good future candidate:

```
EXAMPLE / FLOW
GET /api/v1/reports/campaigns
```

if report data can tolerate a short cache duration.

Never Cache Across Security Boundaries Incorrectly

A cached response for:

```
EXAMPLE / FLOW
Tenant 1
```

must never be returned to:

```
EXAMPLE / FLOW
Tenant 2.
```

Cache keys must include security-relevant variation.

Pagination

All potentially large collections need pagination.

Examples:

```
EXAMPLE / FLOW
GET /constituents

GET /gifts

GET /payments

GET /settlements

GET /reports/top-donors
```

Request:

EXAMPLE / FLOW

```
?pageSize=50
&continuationToken=...
```

or initially:

EXAMPLE / FLOW

```
?page=1
&pageSize=50
```

• Pagination DTO

EXAMPLE / FLOW

```
public sealed record PageRequest(
    int PageSize,
    string? ContinuationToken);
```

Response:

EXAMPLE / FLOW

```
public sealed record PagedResult<T>(
    IReadOnlyList<T> Items,
    string? NextToken);
```

• Page Limits

Never let clients send:

EXAMPLE / FLOW

```
pageSize = 10,000,000
```

Enforce:

EXAMPLE / FLOW

```
default = 50
maximum = 200
```

or appropriate project values.

• API Performance for 1,000+ Users

You cannot guarantee "supports 1,000 users" from code structure alone.

Performance depends on:

EXAMPLE / FLOW

```
concurrent requests

database size

request type

Azure compute tier

SQL tier

network latency

external providers

cache hit rate

query efficiency
```

But the architecture should follow these rules.

• Rule — Async I/O Everywhere

Use asynchronous APIs for:

EXAMPLE / FLOW

```
SQL Server

HTTP

Service Bus

Blob storage

Key Vault
```

This improves server scalability for I/O-heavy workloads.

• Rule — Do Not Task.Run Database Calls

Bad:

EXAMPLE / FLOW

```
await Task.Run(() =>
    db.Gifts.ToList());
```

ASP.NET already uses Thread Pool threads.

This adds scheduling overhead without making synchronous database I/O properly asynchronous.

Rule — DbContext Is Scoped

One request:

EXAMPLE / FLOW

```
one FundraisingDbContext
```

for normal unit-of-work behavior.

Never store DbContext in:

EXAMPLE / FLOW

```
static field
singleton
```

and never share one DbContext between concurrent threads.

Rule — AsNoTracking for Reads

Especially list/report queries.

Rule — Projection

Select DTO fields in SQL rather than loading entire object graphs.

Rule — Avoid Lazy Loading

For a learning enterprise API, I recommend avoiding implicit lazy loading.

It can make this code:

EXAMPLE / FLOW

```
foreach (var constituent in constituents)
{
    var gifts = constituent.Gifts;
}
```

quietly execute many extra SQL queries.

Use explicit query design.

Rule — Avoid Huge Object Graphs

Do not:

EXAMPLE / FLOW

```
GET Constituent
→ include all Gifts
→ include all Allocations
→ include all Funds
→ include all Interactions
→ include all Tasks
```

for one endpoint.

Use separate endpoints and pagination.

• Rule — Cache Carefully

Use Redis/output cache later for safe repeated reads.

Do not cache every response.

• Rule — Connection Pooling

ADO.NET/SQL client connection pooling allows requests to reuse underlying database connections.

Do not keep a database connection manually open for the lifetime of the web application.

Open late and dispose promptly.

Pooling manages reuse.

• Rule — Do Not Parallelize Every Query

Three SQL calls in `Task.WhenAll` may reduce one request's latency while tripling concurrent database pressure.

Measure first.

• Rule — ReportingDb for Heavy Analytics

Do not run five-year analytical scans against FundraisingDb for every dashboard refresh.

Use:

EXAMPLE / FLOW

```
ReportingDb
```

for substantial analytics.

• Rule — Rate Limit Expensive Operations

Especially:

EXAMPLE / FLOW

```
report exports
search
payment initiation
```

• Rule — Observe Before Optimizing

Use:

EXAMPLE / FLOW

```
Application Insights later

OpenTelemetry

SQL Query Store

SQL execution plans

logs

metrics
```

rather than guessing.

• Logging

Use:

EXAMPLE / FLOW

```
ILogger<T>
```

Example:

EXAMPLE / FLOW

```
_logger.LogInformation(
    "Gift {GiftId} created for tenant {TenantId}",
    giftId,
    tenantId);
```

Use structured logging.

Do not do:

EXAMPLE / FLOW

```
_logger.LogInformation(
    $"Gift {giftId} created...");
```

if structured templates are available.

Never Log Sensitive Data

Do not log:

EXAMPLE / FLOW

```
access tokens

refresh tokens

full card details

provider secrets

passwords

raw private keys
```

Be careful with:

EXAMPLE / FLOW

```
donor PII
payment payloads
```

as well.

Correlation / Trace IDs

One operation may travel:

EXAMPLE / FLOW

```
Angular
   |
   v
Fundraising API
   |
   v
Service Bus
   |
   v
Payment API
   |
   v
Payment Provider
```

You need to correlate logs.

Use distributed tracing later with:

EXAMPLE / FLOW

```
OpenTelemetry
Application Insights
```

so one business request can be followed through multiple components.

Health Checks

Expose health endpoints such as:

EXAMPLE / FLOW

```
/health/live

/health/ready
```

Liveness:

EXAMPLE / FLOW

```
Is process alive?
```

Readiness:

EXAMPLE / FLOW

```
Is application ready to receive work?
```

Do not put expensive full business queries into every health check.

Design Patterns — Which Ones This Project Actually Needs

Pattern	Where to Use
Clean Architecture	overall solution
Dependency Injection	all services
CQRS	commands vs queries
Repository	aggregate-specific DB abstraction where useful
Unit of Work	DbContext/local transaction
Factory	choose payment provider
Strategy	payment/fraud behavior variants
Adapter	wrap external provider SDK
Decorator	metrics/logging/caching cross-cutting behavior
Specification	optional reusable query rules
Result	expected application outcomes
Options	strongly typed configuration
Outbox	reliable service events
Idempotency	payment/webhook duplicate protection
Retry	transient external failure
Circuit Breaker	unhealthy external services

Pattern	Where to Use
Builder	test-object creation
Singleton	DI lifetime only where shared thread-safe service fits

Do not create a pattern only to say:

EXAMPLE / FLOW

```
"I used every GoF pattern."
```

Patterns solve problems.

• Dynamic / Extensible Payment Design

You said you want it to work dynamically.

This is a good example.

Define:

EXAMPLE / FLOW

```
public interface IPaymentGateway
{
    string ProviderName { get; }

    Task<GatewayPaymentResult> CreatePaymentAsync(
        GatewayPaymentRequest request,
        CancellationToken cancellationToken);
}
```

Implement:

EXAMPLE / FLOW

```
ProviderAPaymentGateway
ProviderBPaymentGateway
```

Factory receives all providers:

EXAMPLE / FLOW

```
public sealed class PaymentGatewayFactory
{
    private readonly IReadOnlyDictionary<
        string,
        IPaymentGateway> _gateways;

    public PaymentGatewayFactory(
        IEnumerable<IPaymentGateway> gateways)
    {
        _gateways = gateways.ToDictionary(
            x => x.ProviderName,
            StringComparer.OrdinalIgnoreCase);
    }

    public IPaymentGateway Get(string providerName)
```

```
    {
        return _gateways.TryGetValue(
            providerName,
            out var gateway)
            ? gateway
            : throw new UnsupportedPaymentProviderException(
                providerName);
    }
}
```

Now adding Provider C requires:

EXAMPLE / FLOW

```
new implementation
+
DI registration
```

rather than rewriting the entire payment workflow.

That is:

EXAMPLE / FLOW

```
abstraction
+
polymorphism
+
factory
+
DI
```

working together.

• Fundraising API Endpoints

Constituents

EXAMPLE / FLOW

```
POST   /api/v1/constituents

GET    /api/v1/constituents/{id}

PUT    /api/v1/constituents/{id}

GET    /api/v1/constituents

GET    /api/v1/constituents/{id}/summary

GET    /api/v1/constituents/{id}/timeline

GET    /api/v1/constituents/{id}/gifts

GET    /api/v1/constituents/{id}/interactions

GET    /api/v1/constituents/{id}/tasks
```

Create Constituent — .NET Topics Used

EXAMPLE / FLOW

```
Controller
DTO
record
validation
async/await
CancellationToken
Scoped DbContext
EF Core
OOP entity
DI
TenantContext
Result
ProblemDetails
xUnit
```

Flow:

EXAMPLE / FLOW

```
POST constituent
      |
      v
CreateConstituentRequest
      |
      v
validation
      |
      v
CreateConstituentCommand
      |
      v
Handler
      |
      v
Domain Constituent
      |
      v
DbContext
      |
      v
SaveChangesAsync
      |
      v
201 Created
```

Update Constituent

Topics:

EXAMPLE / FLOW

```
PUT
rowversion
optimistic concurrency
DbUpdateConcurrencyException
409 Conflict
ProblemDetails
```

Constituent Search

Topics:

EXAMPLE / FLOW

```
GET
LINQ
IQueryable
Where
Select
OrderBy
AsNoTracking
pagination
SQL indexes
async
```

Constituent Timeline

Topics:

EXAMPLE / FLOW

```
Dapper
stored procedure/query
UNION ALL in SQL
async
projection
pagination
```

Campaign Endpoints

EXAMPLE / FLOW

```
GET    /api/v1/campaigns

POST   /api/v1/campaigns

GET    /api/v1/campaigns/{id}

PUT    /api/v1/campaigns/{id}

GET    /api/v1/campaigns/{id}/performance

GET    /api/v1/campaigns/{id}/history

GET    /api/v1/campaigns/{id}/as-of
```

History endpoint teaches:

EXAMPLE / FLOW

```
Temporal SQL
Dapper/EF raw query
DTO projection
```

Gift Endpoints

EXAMPLE / FLOW

```
POST /api/v1/gifts

GET /api/v1/gifts/{id}

PUT /api/v1/gifts/{id}

GET /api/v1/gifts

GET /api/v1/gifts/{id}/allocations
```

POST Gift — Most Important Fundraising Endpoint

Use:

EXAMPLE / FLOW

```
Controller

CreateGiftRequest

List<GiftAllocationRequest>

HashSet<long>
for duplicate Fund validation

CreateGiftCommand

async/await

CancellationToken

stored procedure or EF transaction

Outbox

Result

201 Created
```

Do not call Payment provider inside the FundraisingDb transaction.

Instead:

EXAMPLE / FLOW

```
Gift committed
+
Outbox event
```

then Payment workflow begins asynchronously.

Task Endpoints

EXAMPLE / FLOW

```
POST /api/v1/tasks

GET /api/v1/tasks

GET /api/v1/tasks/open

GET /api/v1/tasks/overdue

PUT /api/v1/tasks/{id}

PUT /api/v1/tasks/{id}/complete
```

Topics:

EXAMPLE / FLOW

```
filtered SQL index

LINQ

AsNoTracking

RowVersion

authorization

async
```

• Payments API Endpoints

EXAMPLE / FLOW

```
POST /api/v1/payments

GET /api/v1/payments/{id}

GET /api/v1/payments/{id}/attempts

POST /api/v1/payments/{id}/refunds

GET /api/v1/payments/{id}/refunds

POST /api/v1/payment-provider/webhooks

GET /api/v1/settlements

GET /api/v1/settlements/{id}
```

• POST Payment

This endpoint uses many advanced patterns.

EXAMPLE / FLOW

```
Authentication / service authorization

Idempotency-Key

CreatePaymentCommand

Idempotency service

PaymentGatewayFactory

Strategy/Adapter

HttpClient

async/await

timeout
```

```
resilience

local SQL transaction

Outbox

logging

structured errors
```

Payment Creation Flow

EXAMPLE / FLOW

```
Request
   |
   v
Validate identity
   |
   v
Validate Idempotency-Key
   |
   v
Check/create idempotency record
   |
   v
Create Payment
   |
   v
Choose payment provider
   |
   v
Call provider
   |
   v
Update Payment state
   |
   v
Return response
```

Be careful with where transaction boundaries occur around external HTTP calls.

Do not hold SQL locks while waiting unnecessarily on a payment provider.

Webhook Endpoint

EXAMPLE / FLOW

```
POST /api/v1/payment-provider/webhooks
```

This is different from normal human-user endpoints.

It should perform:

EXAMPLE / FLOW

```
provider authentication/signature validation

read provider event ID

insert WebhookEvent

database uniqueness check

idempotent processing

return provider-expected status
```

Do not require normal Angular user authentication for a provider webhook.

Use the provider's webhook-security mechanism.

• Refund Endpoint

EXAMPLE / FLOW

```
POST /api/v1/payments/{paymentId}/refunds
```

Requires:

EXAMPLE / FLOW

```
Finance authorization policy

request validation

remaining-refundable-amount validation

concurrency control

provider integration

idempotency

audit logging
```

• Reporting API

Reporting is read-mostly.

Endpoints:

EXAMPLE / FLOW

```
GET /api/v1/reports/executive

GET /api/v1/reports/giving/monthly

GET /api/v1/reports/campaigns

GET /api/v1/reports/funds

GET /api/v1/reports/donors/top

GET /api/v1/reports/payments/performance

GET /api/v1/reports/payments/refunds

GET /api/v1/reports/interactions
```

Use:

EXAMPLE / FLOW

```
AsNoTracking
Dapper
projection
async
pagination
output caching where safe
authorization
```

No normal:

EXAMPLE / FLOW

```
POST FactGift
```

endpoint should exist for Angular.

Reporting data is populated through ETL/integration.

Complete Topic-to-Project Map

.NET Topic	Project Use
Classes	all entities/services
Objects	Gifts, Payments, etc.
Encapsulation	Gift/Payment state changes
Abstraction	payment gateway, repositories
Interface	infrastructure ports
Inheritance	exception base types, limited shared behavior
Polymorphism	payment-provider implementations
Composition	aggregates and services

.NET Topic	Project Use
Generics	`Result<T>`, `PagedResult<T>`
List	Gift allocations
HashSet	duplicate Fund validation
Dictionary	provider lookup
ConcurrentDictionary	thread-safe process cache only if needed
Queue	learning/in-process only
LINQ	EF queries
IEnumerable	sequences
IQueryable	EF query composition
IAsyncEnumerable	streaming where justified
async/await	all I/O
Task	asynchronous operations
Task.WhenAll	independent I/O only
Parallel.ForEachAsync	bounded independent work, not normal EF CRUD
CancellationToken	every significant async request
static	constants/extensions/pure helpers
Transient	lightweight stateless services
Scoped	DbContext, tenant context, request services
Singleton	thread-safe shared services
Configuration	appsettings/env/Key Vault
Options	strongly typed settings
Controllers	main public Web APIs
DTOs	API contracts
API Versioning	breaking contract evolution
OpenAPI	API documentation
EF Core	CRUD/unit of work
Dapper	tuned SQL/reporting/SP calls
Repository	aggregate-specific abstraction
Unit of Work	DbContext/local transaction
CQRS	commands vs queries
Factory	payment-provider selection
Strategy	payment/fraud algorithms

.NET Topic	Project Use
Adapter	third-party provider integration
Decorator	logging/metrics/caching
Custom Exceptions	meaningful exceptional conditions
ProblemDetails	standard HTTP errors
OAuth 2.0	API authorization
OIDC	user sign-in
JWT	API access token format
SAML	enterprise federation concept
Claims	user/tenant/security data
Roles	business roles
Policies	business authorization
HttpClient	external provider
Retry	transient provider errors
Circuit breaker	provider failure protection
BackgroundService	Outbox processing
Idempotency	Payment/Webhook safety
Output Cache	safe reporting reads
Rate Limiting	abuse/load protection
ILogger	structured logs
OpenTelemetry	traces/metrics
xUnit	unit/integration tests

xUnit Testing

Testing must be part of the architecture, not something added after the entire project is completed.

Use:

EXAMPLE / FLOW

```
Unit Tests

Application Tests

Integration Tests

API Tests

Database Tests

Security Tests
```

• Domain Unit Test

Example:

> Gift should reject an allocation greater than allowed business rules.

JSON

```
[Fact]
public void AddAllocation_WithNegativeAmount_ShouldFail()
{
    // Arrange

    // Act

    // Assert
}
```

No:

EXAMPLE / FLOW

```
SQL Server
HTTP
Azure
```

should be required for a pure domain unit test.

• Application Handler Test

Test:

EXAMPLE / FLOW

```
CreateGiftHandler
```

with fake/test implementations of:

EXAMPLE / FLOW

```
IGiftWriter
ITenantContext
```

Verify:

EXAMPLE / FLOW

```
correct tenant passed

validation performed

result returned
```

Integration Test

An integration test should exercise real infrastructure boundaries.

Example:

EXAMPLE / FLOW

```
API
   |
   v
Application
   |
   v
EF Core
   |
   v
real SQL Server test database
```

For meaningful SQL integration tests, prefer an actual SQL Server environment/container rather than assuming EF's in-memory provider behaves identically to SQL Server.

WebApplicationFactory

ASP.NET Core integration testing can run the application in a test host.

Concept:

EXAMPLE / FLOW

```
xUnit
   |
   v
WebApplicationFactory
   |
   v
HTTP request
   |
   v
real API pipeline
```

Then test:

EXAMPLE / FLOW

```
var response =
    await client.GetAsync(
        "/api/v1/constituents/1001");
```

Endpoint Test Cases

For every important endpoint test:

EXAMPLE / FLOW

```
success

validation error

not found

unauthenticated

unauthorized

concurrency conflict

database failure behavior where practical
```

Payment Tests

Critical cases:

EXAMPLE / FLOW

```
same idempotency key twice
→ one Payment

same webhook twice
→ one processing action

refund > remaining amount
→ rejected

provider timeout
→ known error behavior

provider failure
→ correct Payment status
```

Concurrency Integration Test

Create two contexts.

Both load:

EXAMPLE / FLOW

```
Task RowVersion = X
```

Context A saves.

Context B saves.

Expected:

EXAMPLE / FLOW

```
DbUpdateConcurrencyException
```

and API maps to:

EXAMPLE / FLOW

```
409
```

This proves your concurrency design actually works.

Architecture Tests

Ensure:

EXAMPLE / FLOW

```
Domain
does not reference Infrastructure

Application
does not reference API

Payment domain
does not reference Fundraising infrastructure
```

These can be automated later with architecture-testing libraries or reflection-based tests.

Test Data Builder Pattern

Tests often need objects.

Instead of repeating:

EXAMPLE / FLOW

```
new Gift(...)
```

with twenty values, create:

EXAMPLE / FLOW

```
GiftBuilder
```

Example:

EXAMPLE / FLOW

```
var gift = new GiftBuilder()
    .WithAmount(1000m)
    .WithConstituent(1001)
    .Build();
```

That is an excellent practical use of the Builder pattern.

• Unit Test Naming

Use names that explain behavior.

Example:

EXAMPLE / FLOW

```
CreateGift_WhenAllocationsDoNotEqualTotal_ReturnsValidationFailure
```

rather than:

EXAMPLE / FLOW

```
TestGift1
```

• What NOT to Unit Test

Do not waste time testing:

EXAMPLE / FLOW

```
automatic property getter

framework implementation

whether List<T>.Add works
```

Test your behavior.

• Dynamic Architecture Rule

You wanted this project to remain dynamic and extensible.

The correct way is not:

EXAMPLE / FLOW

```
reflection everywhere
dynamic keyword everywhere
huge switch statements
```

Instead use:

```
EXAMPLE / FLOW
interfaces
DI
strategies
configuration
factories
policies
handlers
```

Example:

Today:

```
EXAMPLE / FLOW
Provider A
```

Tomorrow:

```
EXAMPLE / FLOW
Provider B
```

should require:

```
EXAMPLE / FLOW
new provider implementation
configuration
registration
```

rather than changing every Payment controller and service.

• `dynamic` Keyword

C# also has a `dynamic` keyword.

Example:

```
EXAMPLE / FLOW
dynamic value = GetSomething();
```

Compile-time type checks are delayed.

Do **not** use this as the normal architecture strategy.

It reduces compile-time safety.

Useful only in specialized interop/dynamic scenarios.

For normal APIs use:

```
EXAMPLE / FLOW
strongly typed DTOs
generics
interfaces
```

• Performance-Oriented Endpoint Design

Bad

EXAMPLE / FLOW

```
GET /api/constituents
```

returns:

EXAMPLE / FLOW

```
all 5 million Constituents
+
all Gifts
+
all Tasks
+
all Interactions
```

• Better

EXAMPLE / FLOW

```
GET /api/v1/constituents?pageSize=50
```

returns summary rows.

Then:

EXAMPLE / FLOW

```
GET /api/v1/constituents/1001/gifts
```

loads Gifts separately.

Then:

EXAMPLE / FLOW

```
GET /api/v1/constituents/1001/interactions
```

loads interactions separately.

This reduces:

EXAMPLE / FLOW

```
SQL work

response size

memory

Angular rendering

network traffic
```

Avoid Repeated Database Calls

Bad handler:

EXAMPLE / FLOW

```
Check Constituent

Check Campaign

Check Fund 1

Check Fund 2

Check Fund 3
```

as five separate sequential round trips if one set-based query/procedure can validate all needed records.

Let SQL Server perform set-oriented operations.

API Response Compression

Large JSON responses may benefit from HTTP response compression.

Use it for appropriate text/JSON traffic.

Do not use compression as an excuse to return unnecessarily large payloads.

Streaming

For very large exports, consider streaming rather than:

EXAMPLE / FLOW

```
load 2 GB into List<T>
```

before writing response.

But this is an advanced specialized endpoint, not normal CRUD.

Memory Management

Avoid retaining:

EXAMPLE / FLOW

```
huge lists

large byte arrays

global caches

DbContexts
```

longer than necessary.

ASP.NET Core's GC manages memory, but your code still determines how much memory becomes live.

Startup Validation

If configuration is wrong:

EXAMPLE / FLOW
```
PaymentProvider:BaseUrl missing
```

it is better for the application to fail at startup than wait until the first donor payment.

Use:

EXAMPLE / FLOW
```
ValidateOnStart()
```

for critical options. ASP.NET Core's Options system supports startup validation directly.

Secrets

Development:

EXAMPLE / FLOW
```
User Secrets
```

Production later:

EXAMPLE / FLOW
```
Managed Identity
+
Azure Key Vault
```

Do not put:

EXAMPLE / FLOW
```
SQL passwords
payment provider secret
client secrets
```

in source control.

Fundraising.API Suggested Structure

EXAMPLE / FLOW
```
Fundraising.Api/
│
├── Controllers/
│   ├── ConstituentsController.cs
│   ├── CampaignsController.cs
```

```
│   ├── FundsController.cs
│   ├── GiftsController.cs
│   ├── InteractionsController.cs
│   └── TasksController.cs
│
├── Authentication/
│
├── Authorization/
│
├── ExceptionHandling/
│
├── Middleware/
│
├── OpenApi/
│
├── Versioning/
│
├── Extensions/
│
├── appsettings.json
├── appsettings.Development.json
└── Program.cs
```

• Fundraising.Application Structure

EXAMPLE / FLOW

```
Fundraising.Application/
│
├── Abstractions/
│   ├── Persistence/
│   ├── Identity/
│   ├── Messaging/
│   └── Time/
│
├── Constituents/
│   ├── Create/
│   ├── Update/
│   ├── GetById/
│   ├── Search/
│   ├── Summary/
│   └── Timeline/
│
├── Campaigns/
│
├── Funds/
│
├── Gifts/
│   ├── Create/
│   ├── Update/
│   ├── GetById/
│   └── Search/
│
├── Interactions/
│
└── Tasks/
```

This is feature-oriented rather than putting:

EXAMPLE / FLOW

```
all commands
all handlers
all DTOs
```

into giant global folders.

Fundraising.Infrastructure

EXAMPLE / FLOW

```
Fundraising.Infrastructure/
│
├── Persistence/
│   ├── FundraisingDbContext.cs
│   ├── Configurations/
│   ├── Repositories/
│   └── Queries/
│
├── StoredProcedures/
│
├── Messaging/
│   └── Outbox/
│
├── Identity/
│
└── DependencyInjection.cs
```

Payments Solution Structure

EXAMPLE / FLOW

```
Payments.Api/
│
├── Controllers/
│   ├── PaymentsController.cs
│   ├── RefundsController.cs
│   ├── WebhooksController.cs
│   └── SettlementsController.cs
│
└── ...

Payments.Application/
│
├── Payments/
├── Refunds/
├── Webhooks/
├── Settlements/
└── Abstractions/

Payments.Infrastructure/
│
├── Persistence/
├── PaymentProviders/
│   ├── ProviderA/
│   └── ProviderB/
```

```
├── Messaging/
└── Security/
```

Reporting Structure

EXAMPLE / FLOW

```
Reporting.Api/
│
└── Controllers/
    └── ReportsController.cs

Reporting.Application/
│
├── Executive/
├── Giving/
├── Campaigns/
├── Funds/
├── Donors/
├── Payments/
└── Interactions/

Reporting.Infrastructure/
│
├── Persistence/
├── Dapper/
└── ETL/
```

Reporting does not need a complex Domain model if it is primarily read/query oriented.

Clean Architecture should adapt to the problem rather than force identical layers everywhere.

Build Order for the .NET Project

Do this in stages.

Phase 1 — C# Foundation

Before the full Web API, practice using project objects:

EXAMPLE / FLOW

```
classes

constructors

properties

methods

encapsulation

interfaces

inheritance

polymorphism
```

```
composition

records

generics

collections

LINQ

exceptions

async/await
```

Use:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Payment
Refund
```

for exercises.

Phase 2 — Solution and Clean Architecture

Create:

EXAMPLE / FLOW

```
solution

Domain projects

Application projects

Infrastructure projects

API projects

test projects
```

Set project references correctly.

Phase 3 — FundraisingDbContext

Map:

EXAMPLE / FLOW

```
Tenant
AppUser
Constituent
Address
ConstituentAddress
Campaign
Fund
Gift
GiftAllocation
Interaction
Task
OutboxMessage
```

Connect to existing SQL Server schema.

• Phase 4 — Basic CRUD

Implement first:

EXAMPLE / FLOW

```
Constituent CRUD

Campaign CRUD

Fund CRUD

Task CRUD
```

Learn:

EXAMPLE / FLOW

```
Controllers

DTOs

DI

Scoped DbContext

LINQ

async/await

CancellationToken

validation

ProblemDetails

xUnit
```

Phase 5 — Gift Workflow

Implement:

EXAMPLE / FLOW

```
Create Gift

Gift allocations

SQL transaction / stored procedure

Outbox

RowVersion

Gift search

Gift detail
```

This is your main Fundraising use case.

Phase 6 — Authentication / Authorization

Add:

EXAMPLE / FLOW

```
Microsoft Entra ID

OAuth 2.0

OIDC

JWT Bearer validation

claims

roles

policies

TenantContext
```

Then protect every endpoint.

Phase 7 — API Versioning / OpenAPI

Create:

EXAMPLE / FLOW

```
/api/v1/...
```

Generate OpenAPI.

Document:

EXAMPLE / FLOW

```
security

responses

models
```

Phase 8 — Payment API

Build:

EXAMPLE / FLOW

```
PaymentDbContext

Payment creation

payment provider abstraction

Factory

Strategy

Adapter

HttpClient

idempotency

webhook

refund

settlement
```

Phase 9 — Reporting API

Implement:

EXAMPLE / FLOW

```
Dapper

AsNoTracking

warehouse queries

pagination

output caching
```

Phase 10 — Background Processing

Implement:

EXAMPLE / FLOW

```
OutboxPublisher

WebhookProcessor

background worker

scope creation

bounded batch processing

retry/error tracking
```

• Phase 11 — Performance

Test:

EXAMPLE / FLOW

```
AsNoTracking

projection

N+1 prevention

pagination

keyset pagination

output caching

rate limiting

connection pooling

async end-to-end
```

Then perform load testing later.

• Phase 12 — Complete Tests

Every important workflow gets:

EXAMPLE / FLOW

```
domain tests

application tests

integration tests

API tests

authorization tests

concurrency tests
```

Final .NET Completion Checklist

Before moving to Azure/microservices, you should be able to demonstrate:

EXAMPLE / FLOW ✓ C# classes and objects

✓ Encapsulation

✓ Abstraction

✓ Interfaces

✓ Inheritance

✓ Polymorphism

✓ Composition

✓ SOLID

✓ Records

✓ Generics

✓ List<T>

✓ HashSet<T>

✓ Dictionary<TKey,TValue>

✓ Concurrent collections concept

✓ IEnumerable<T>

✓ IQueryable<T>

✓ IAsyncEnumerable<T>

✓ LINQ

✓ async / await

✓ Task

✓ Task.WhenAll

✓ CancellationToken

✓ Parallelism differences

✓ DbContext thread-safety understanding

✓ static

✓ Dependency Injection

✓ Transient

- ✓ Scoped
- ✓ Singleton
- ✓ Options
- ✓ appsettings
- ✓ Program.cs
- ✓ Clean Architecture
- ✓ Domain Layer
- ✓ Application Layer
- ✓ Infrastructure Layer
- ✓ API Layer
- ✓ Controllers
- ✓ DTOs
- ✓ Validation
- ✓ HTTP methods
- ✓ HTTP status codes
- ✓ API versioning
- ✓ OpenAPI / Swagger
- ✓ EF Core
- ✓ Dapper
- ✓ Stored procedure calls
- ✓ SQL transactions
- ✓ RowVersion concurrency
- ✓ Repository pattern
- ✓ Unit of Work
- ✓ CQRS
- ✓ Mediator concept
- ✓ Factory
- ✓ Strategy
- ✓ Adapter
- ✓ Decorator
- ✓ Specification concept

✓ Result pattern

✓ Outbox

✓ Idempotency

✓ Custom exceptions

✓ Global exception handling

✓ ProblemDetails

✓ HttpClientFactory

✓ Timeout

✓ Retry

✓ Circuit breaker

✓ Authentication

✓ Authorization

✓ OAuth 2.0

✓ OpenID Connect

✓ JWT

✓ Access tokens

✓ ID tokens

✓ SAML concepts

✓ Microsoft Entra ID

✓ Claims

✓ Roles

✓ Policies

✓ scopes

✓ Tenant authorization

✓ CORS

✓ Rate limiting

✓ Caching

✓ Output caching

✓ BackgroundService

✓ Logging

✓ Correlation IDs

✓ OpenTelemetry concepts

✓ Health checks

✓ xUnit

✓ Unit testing

✓ Integration testing

✓ API testing

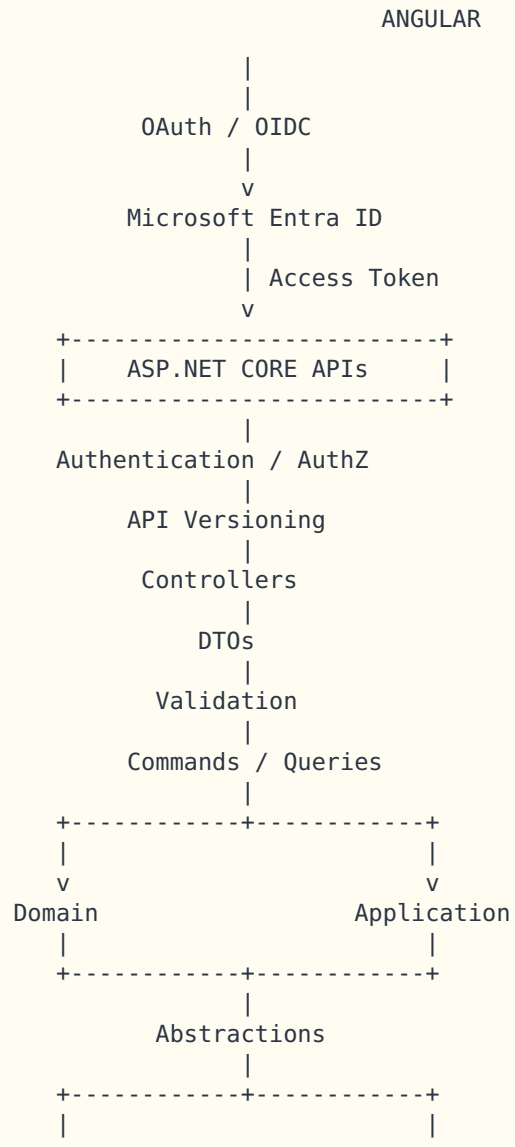
✓ Concurrency testing

✓ Architecture testing

Final .NET Architecture

The complete request architecture should eventually look like this:

EXAMPLE / FLOW



```
                  v                       v
               EF Core /               Dapper /
               SQL CRUD              Stored Procedures
                  |                       |
                  +-----------+-----------+
                              |
            +-----------------+-----------------+
            |                 |                 |
            v                 v                 v
      FundraisingDb       PaymentDb        ReportingDb
            |
            v
      OutboxMessage
            |
            v
      Background Worker
            |
            v
      Service Bus later


Payment API
     |
     v
IPaymentGateway
     |
     v
PaymentGatewayFactory
     |
     +---------+---------+
     |                   |
     v                   v
Provider A Adapter   Provider B Adapter
     |
     v
HttpClient
     |
     v
External Payment Provider
```

And the runtime concerns surround everything:

EXAMPLE / FLOW

```
ProblemDetails
Logging
OpenTelemetry
Rate Limiting
Caching
Health Checks
Configuration
Key Vault later
xUnit tests
```

The most important .NET rule for this project is:

Keep controllers thin, keep domain rules close to the domain, put use-case orchestration in the Application layer, put SQL/external-service implementation in Infrastructure, make every I/O path asynchronous, keep DbContext scoped and never use it concurrently, use interfaces and DI for replaceable dependencies, use patterns only when they solve a real problem, standardize errors with ProblemDetails, secure APIs with standards-based OAuth/OIDC access tokens and authorization policies, and measure performance before introducing unnecessary complexity.

For the high-user-load goal, remember this shorter rule:

EXAMPLE / FLOW

```
Async I/O
+
small HTTP payloads
+
pagination
+
projection
+
AsNoTracking reads
+
short SQL transactions
+
good SQL indexes
+
no N+1
+
no shared DbContext
+
bounded parallelism
+
cache only where safe
+
rate limiting
+
Query Store
+
observability
```

That is the .NET foundation you should complete before we move the same platform into the **Azure / GitHub / CI-CD / Docker / Kubernetes / Service Bus / Functions / load balancing / deployment / microservices** document.